HybridRAG: Implementation Report

Integrating Knowledge Graphs and Vector Retrieval for
Enhanced Information Extraction from Financial Documents

Extended with Custom Ontology-Based Knowledge Graph Construction

Bibek Singh Dhody

February 2026

Abstract

This report presents the implementation of HybridRAG, a Retrieval-Augmented Generation system that combines Knowledge Graphs with Vector-based retrieval for enhanced question answering over financial documents. We describe the complete system architecture, implementation details, and present an extension that enables custom ontology-based knowledge graph construction using Large Language Models (LLMs). The extension allows domain experts to define their own entity types and relationship schemas, which are then used by an LLM (Ollama) to intelligently extract structured knowledge from unstructured text. This approach significantly improves domain-specific information extraction compared to traditional Named Entity Recognition (NER) methods.

Contents

1 Introduction

1.1 Background and Motivation

Financial documents such as SEC filings, earnings reports, and annual reports contain rich structured and unstructured information. Traditional Retrieval-Augmented Generation (RAG) systems rely solely on vector similarity search, which often fails to capture the complex relationships between entities mentioned in these documents.

HybridRAG addresses this limitation by combining:

- **Vector RAG**: Semantic similarity-based retrieval using dense embeddings
- **Graph RAG**: Structured knowledge retrieval from Knowledge Graphs
- **Hybrid Fusion**: Intelligent aggregation of both retrieval methods

1.2 Problem Statement

The key challenges addressed by this implementation include:

1. Extracting structured knowledge from unstructured financial documents
2. Building domain-specific knowledge graphs with custom entity and relationship types
3. Combining structured (graph) and unstructured (vector) retrieval methods
4. Enabling domain experts to define custom ontologies without programming expertise

1.3 Contributions

This implementation makes the following contributions:

1. A complete Java-based HybridRAG system with Vector RAG, Graph RAG, and Hybrid fusion
2. Integration with multiple LLM providers (Ollama, OpenAI, Gemini)
3. **Extension**: A novel LLM-based knowledge extraction pipeline that uses custom ontologies to intelligently extract domain-specific entities and relationships

2 System Architecture

2.1 Overall Architecture

The HybridRAG system consists of several interconnected components as shown in Figure 1.

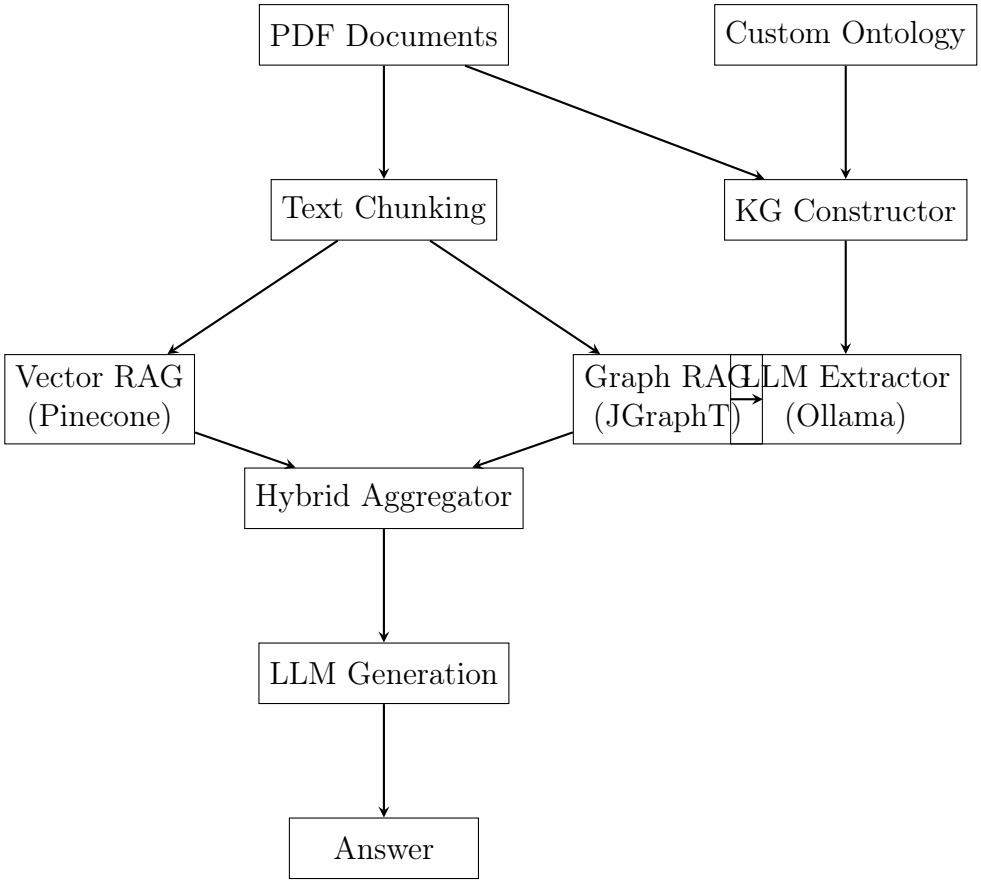


Figure 1: HybridRAG System Architecture

2.2 Component Overview

2.2.1 Vector RAG Component

The Vector RAG component handles semantic similarity-based retrieval:

- **Embedding Generation**: Uses Gemini API for generating 768-dimensional embeddings
- **Vector Storage**: Pinecone vector database for efficient similarity search
- **Retrieval**: Top-K retrieval based on cosine similarity

2.2.2 Graph RAG Component

The Graph RAG component handles structured knowledge retrieval:

- **Graph Storage**: JGraphT library for in-memory graph operations
- **Entity Matching**: Fuzzy matching to find relevant entities in queries
- **Subgraph Extraction**: DFS-based traversal to extract relevant triplets

2.2.3 Knowledge Graph Constructor

The KG Constructor supports two extraction modes:

- **NER Mode**: Stanford CoreNLP-based Named Entity Recognition
- **LLM Mode**: Custom ontology-driven extraction using Ollama

3 Implementation Details

3.1 Technology Stack

Table 1: Technology Stack

Component	Technology	Purpose
Programming Language	Java 17	Core implementation
Build Tool	Maven	Dependency management
PDF Processing	Apache PDFBox	Text extraction from PDFs
NLP	Stanford CoreNLP	Named Entity Recognition
Graph Library	JGraphT	Knowledge Graph operations
Vector Database	Pinecone	Embedding storage and retrieval
LLM (Local)	Ollama (llama3.1:8b)	Text generation and extraction
LLM (Cloud)	Gemini / OpenAI	Embeddings and generation

3.2 Project Structure

```
src/main/java/com/hybridrag/
          Main.java                      # CLI entry point
          config/
                HybridRAGConfig.java     # Configuration management
          model/
                CustomOntology.java      # Ontology schema definition
                Document.java            # Document representation
                DocumentChunk.java       # Text chunk model
                Entity.java              # Entity model
                Relationship.java        # Relationship model
                RAGResult.java           # Query result model
          service/
              VectorRAG.java             # Vector retrieval
              GraphRAG.java              # Graph retrieval
              HybridRAG.java             # Hybrid aggregation
              KnowledgeGraphConstructor.java  # KG building
              LLMKnowledgeExtractor.java      # LLM-based extraction
              OllamaService.java         # Ollama API client
              GeminiService.java         # Gemini API client
              PineconeRestClient.java    # Pinecone API client
```

Listing 1: Project Directory Structure

3.3 Core Data Models

3.3.1 Entity Model

```java
public class Entity {
    private String id;
    private String name;
    private String type;  // e.g., COMPANY, PERSON, PRODUCT
    private Map<String, Object> properties;
}
```

Listing 2: Entity Class

3.3.2 Relationship Model

```java
public class Relationship {
    private String id;
    private String sourceId;
    private String targetId;
    private String type;  // e.g., PRODUCES, LED_BY, LOCATED_IN
    private Map<String, Object> properties;
}
```

Listing 3: Relationship Class

4 Extension: Custom Ontology-Based Knowledge Graph Construction

This section describes the major extension to the original HybridRAG system: the ability to define custom domain ontologies and use LLM-based extraction to build knowledge graphs.

4.1 Motivation

Traditional NER-based knowledge extraction has significant limitations:

- **Fixed Entity Types**: Standard NER models recognize only predefined types (PERSON, ORGANIZATION, LOCATION, etc.)
- **No Domain Specificity**: Cannot extract domain-specific entities like FINANCIAL_METRIC, DRUG, or LEGAL_CONCEPT
- **Limited Relationships**: Dependency parsing captures syntactic relationships, not semantic ones
- **Keyword Matching is Insufficient**: Simple keyword matching cannot understand context or synonyms

4.2 Solution: LLM-Based Ontology-Driven Extraction

Our extension uses Large Language Models to intelligently extract entities and relationships based on a user-defined ontology schema. The key insight is:

> *The ontology defines WHAT to look for (schema), NOT explicit keywords. The LLM does the actual extraction by UNDERSTANDING the text semantically.*

4.3 Custom Ontology Schema

The ontology is defined as a JSON file with the following structure:

```
{
  "name": "Financial Document Ontology",
  "description": "Ontology for financial documents",
  "domain": "finance",
  "entityTypes": [
    {
      "type": "COMPANY",
      "description": "A business organization or corporation",
      "examples": ["Apple Inc.", "Microsoft", "Tesla"]
    },
    {
      "type": "FINANCIAL_METRIC",
      "description": "A quantifiable financial measure",
      "examples": ["revenue", "net income", "EPS"]
    }
  ],
  "relationTypes": [
    {
      "type": "REPORTED",
      "description": "Company reported a financial metric",
      "sourceType": "COMPANY",
      "targetType": "FINANCIAL_METRIC"
    }
  ]
}
```

Listing 4: Custom Ontology Structure

4.4 LLM-Based Extraction Pipeline

Algorithm 1 LLM-Based Knowledge Extraction

Require: Document D, Ontology O
Ensure: Knowledge Graph $G = (E, R)$

$E \leftarrow \emptyset$ ▷ Entity set
$R \leftarrow \emptyset$ ▷ Relationship set
$chunks \leftarrow \text{ChunkDocument}(D, 1500)$ ▷ Split into chunks
for each $chunk$ in $chunks$ **do**
    $prompt \leftarrow \text{BuildExtractionPrompt}(chunk, O)$
    $response \leftarrow \text{CallOllama}(prompt)$
    $(e, r) \leftarrow \text{ParseJSON}(response)$
    $e' \leftarrow \text{ResolveEntities}(e, E)$ ▷ Entity resolution
    $r' \leftarrow \text{ValidateRelationships}(r, O)$ ▷ Validation
    $E \leftarrow E \cup e'$
    $R \leftarrow R \cup r'$
end for
return (E, R)

4.5 Extraction Prompt Design

The extraction prompt is carefully designed to guide the LLM:

```
You are a knowledge extraction system. Extract entities and
relationships from the text according to the ontology.

## Domain: finance

## Entity Types to Extract:
- **COMPANY**: A business organization...
- **FINANCIAL_METRIC**: A quantifiable financial measure...
- **MONEY**: A specific monetary value...

## Relationship Types to Extract:
- **REPORTED** [COMPANY -> FINANCIAL_METRIC]: Company reported...
- **HAS_VALUE** [FINANCIAL_METRIC -> MONEY]: Metric has value...

## Text to analyze:
{chunk_text}

## Output Format:
Return ONLY valid JSON:
{
  "entities": [{"name": "...", "type": "..."}],
  "relationships": [{"source": "...", "target": "...",
                     "type": "..."}]
}
```

Listing 5: LLM Extraction Prompt Template

4.6 Entity Resolution

Entity resolution merges multiple mentions of the same real-world entity:

```java
private Map<String, Entity> resolveEntities(List<Entity> entities) {
    Map<String, Entity> resolved = new HashMap<>();
    for (Entity entity : entities) {
        String canonical = canonicalize(entity.getName());
        if (resolved.containsKey(canonical)) {
            // Merge with existing entity
            Entity existing = resolved.get(canonical);
            existing.addAlias(entity.getName());
        } else {
            resolved.put(canonical, entity);
        }
    }
    return resolved;
}

private String canonicalize(String name) {
    // Remove suffixes like "Inc.", "Corp.", "Ltd."
    // Normalize whitespace and case
    return name.replaceAll("\\s+(Inc|Corp|Ltd)\\.?$", "")
               .trim().toLowerCase();
}
```

Listing 6: Entity Resolution Algorithm

4.7 Relationship Validation

Relationships are validated against the ontology constraints:

```java
public boolean isValidRelationship(String relationType,
                                   String sourceType,
                                   String targetType) {
    for (RelationType rt : relationTypes) {
        if (rt.getType().equals(relationType)) {
            // Check source type constraint
            if (rt.getSourceType() != null &&
                !rt.getSourceType().equals(sourceType)) {
                return false;
            }
            // Check target type constraint
            if (rt.getTargetType() != null &&
                !rt.getTargetType().equals(targetType)) {
                return false;
            }
            return true;
        }
    }
    return false;
}
```

Listing 7: Relationship Validation

5 Supported Domains

The system provides pre-built ontologies for several domains:

5.1 Financial Domain

Table 2: Financial Domain Ontology

Entity Types	**Relationship Types**
COMPANY	REPORTED (Company → Metric)
PERSON	LED_BY (Company → Person)
FINANCIAL_METRIC	PRODUCES (Company → Product)
MONEY	HAS_VALUE (Metric → Money)
PERCENTAGE	LOCATED_IN (Company → Location)
PRODUCT	DURING_PERIOD (* → Date)
DATE	INCREASED / DECREASED
LOCATION	ACQUIRED (Company → Company)

5.2 Medical Domain

Table 3: Medical Domain Ontology

Entity Types	**Relationship Types**
DRUG	TREATS (Drug → Disease)
DISEASE	CAUSES (* → *)
GENE	HAS_SYMPTOM (Disease → Symptom)
SYMPTOM	INHIBITS (Drug → Gene)
TREATMENT	ADMINISTERED_AT (Drug → Dosage)
DOSAGE	

5.3 Legal Domain

Table 4: Legal Domain Ontology

Entity Types	**Relationship Types**
PARTY	SUED_BY (Party → Party)
COURT	PRESIDED_BY (* → Judge)
JUDGE	FILED_IN (* → Court)
CASE_NUMBER	CITED_STATUTE (* → Statute)
STATUTE	RULED_THAT (* → Legal Concept)
LEGAL_CONCEPT	AWARDED (Party → Money)
DATE, MONEY	OCCURRED_ON (* → Date)

6 Experimental Results

6.1 Dataset

We evaluated the system using Apple Inc.'s Q1 2024 10-K filing (80 pages, 273,314 characters).

6.2 Knowledge Graph Statistics

Table 5: Extraction Results from Apple 10-K Document

Metric	Value
Document Pages	80
Total Characters	273,314
Text Chunks Processed	243
Entities Extracted	**987**
MONEY	246
FINANCIAL_METRIC	184
PRODUCT	166
COMPANY	123
DATE	104
LOCATION	74
PERSON	63
PERCENTAGE	27
Relationships Extracted	**518**
PRODUCES	150
LOCATED_IN	106
REPORTED	91
DURING_PERIOD	86
LED_BY	29
DECREASED	21
INCREASED	16
HAS_VALUE	15
ACQUIRED	4

6.3 Sample Query Results

6.3.1 Query 1: Products

```
Question: What products does Apple Inc produce?

Answer: Apple Inc. produces the following products:
1. iPhone              13. Mac mini
2. macOS               14. iMac
3. iOS                 15. iPad
4. App Store           16. MacBook Air
5. Apple Arcade        17. Mac Studio
6. Apple Fitness+      18. watchOS 26
7. Apple Music         19. visionOS 26
8. Apple News+         20. tvOS 26
9. Apple TV 4K         21. Apple Watch Series 11
10. Apple Card         22. Apple Watch SE 3
11. Apple Pay          23. Apple Watch Ultra 3
12. MacBook Pro        24. AirPods
```

6.3.2 Query 2: Leadership and Location

```
Question: Who leads Apple Inc and where is it located?

Answer: Apple Inc. is led by Tim Cook (Timothy D. Cook).
Locations: Cupertino, California, United States, and
international operations in China, India, Japan,
South Korea, Taiwan, Vietnam, Europe (Ireland).
```

7 Usage Guide

7.1 Generating a Custom Ontology

```
# Generate financial ontology (default)
mvn exec:java -Dexec.mainClass="com.hybridrag.Main" \
  -Dexec.args="--mode generate-ontology"

# Generate medical ontology
mvn exec:java -Dexec.mainClass="com.hybridrag.Main" \
  -Dexec.args="--mode generate-ontology --domain medical"

# Generate legal ontology
mvn exec:java -Dexec.mainClass="com.hybridrag.Main" \
  -Dexec.args="--mode generate-ontology --domain legal"
```

Listing 8: Generate Sample Ontology

7.2 Building Knowledge Graph with Custom Ontology

```
# Ensure Ollama is running
ollama run llama3.1:8b

# Build KG with custom ontology
mvn exec:java -Dexec.mainClass="com.hybridrag.Main" \
  -Dexec.args="--mode build-kg-custom \
               --documents data/documents/file.pdf \
               --ontology data/custom_ontology.json"
```

Listing 9: Build Knowledge Graph

7.3 Querying the Knowledge Graph

```
# Query using Graph RAG only
mvn exec:java -Dexec.mainClass="com.hybridrag.Main" \
  -Dexec.args="--mode query-kg \
               --question 'What products does Apple produce?'"

# Query using Hybrid RAG (requires Pinecone)
mvn exec:java -Dexec.mainClass="com.hybridrag.Main" \
  -Dexec.args="--mode query \
               --question 'What was Apple revenue?'"
```

Listing 10: Query Knowledge Graph

8 Comparison: NER vs LLM-Based Extraction

Table 6: Comparison of Extraction Methods

Aspect	**NER-Based**	**LLM-Based (Ours)**
Entity Types	Fixed (7-10 types)	Unlimited (user-defined)
Domain Adaptation	Requires retraining	Schema change only
Relationship Extraction	Syntactic patterns	Semantic understanding
Context Understanding	Limited	Full contextual
Processing Speed	Fast	Slower (API calls)
Setup Complexity	High (CoreNLP)	Low (Ollama)
Customization	Difficult	JSON schema

9 Conclusion and Future Work

9.1 Conclusion

This implementation report presented HybridRAG, a system that combines Knowledge Graphs with Vector Retrieval for enhanced question answering over financial documents. The key contribution is the extension that enables custom ontology-based knowledge graph construction using LLMs.

The LLM-based approach offers several advantages:

- **Flexibility**: Domain experts can define custom entity and relationship types
- **Semantic Understanding**: LLMs understand context and synonyms
- **No Retraining**: New domains require only a schema change
- **Structured Output**: Validated against ontology constraints

9.2 Future Work

1. **Incremental Updates**: Support for adding new documents without rebuilding the entire graph
2. **Graph Embeddings**: Learn entity embeddings for better similarity search
3. **Multi-hop Reasoning**: Support complex queries requiring multiple inference steps
4. **Evaluation Framework**: Comprehensive metrics for extraction quality
5. **UI Interface**: Web-based interface for ontology design and querying

Acknowledgments

This work builds upon the HybridRAG research paper and extends it with custom ontology-based knowledge extraction capabilities.

References

[1] HybridRAG: Integrating Knowledge Graphs and Vector Retrieval Augmented Generation for Efficient Information Extraction.

[2] Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*.

[3] Hogan, A., et al. (2021). Knowledge Graphs. *ACM Computing Surveys*.

[4] Ollama: Run Large Language Models Locally. `https://ollama.ai`

[5] Pinecone: Vector Database for Machine Learning. `https://pinecone.io`

Automated Knowledge Graph Construction Pipeline Using Large Language Models

A Semi-Automated Approach for Ontology and KG Engineering in Cybersecurity Domain

Implementation Report

May 8, 2026

Abstract

This report presents a comprehensive semi-automated pipeline for Knowledge Graph (KG) construction inspired by the methodology proposed in "From human experts to machines: An LLM supported approach to ontology and knowledge graph construction" [1]. Our implementation leverages open-source Large Language Models (LLMs), specifically Cerebras API with Llama 3.1-8B, to minimize human effort in ontology engineering and KG construction. The pipeline encompasses five main stages: (1) Competency Question (CQ) formulation, (2) Ontology creation, (3) Retrieval-Augmented Generation (RAG) for CQ answering, (4) Named Entity Recognition (NER), and (5) Triple extraction with semantic validation. Applied to cybersecurity lab documentation, our pipeline demonstrates the feasibility of automated KG construction with improved quality measures through multi-pass extraction, confidence scoring, and semantic validation.

Recent advances in Large Language Models (LLMs) have revolutionized natural language processing, offering promising opportunities to automate aspects of KG construction. This work explores the semi-automatic construction of KGs for the cybersecurity domain, leveraging open-source LLMs to reduce human effort while maintaining quality.

0.1 Motivation

The cybersecurity domain presents unique challenges for knowledge representation:

- Rapidly evolving attack vectors and defense mechanisms
- Complex relationships between tools, vulnerabilities, and protocols
- Need for reproducibility and provenance tracking in security assessments
- Domain-specific terminology requiring careful handling

Our pipeline addresses these challenges by automating KG construction from cybersecurity lab documentation while ensuring quality through semantic validation and LLM-based evaluation.

0.2 Contributions

This implementation makes the following contributions:

1. **Fully automated pipeline** using only Cerebras API (no proprietary models like GPT-4)
2. **Enhanced triple extraction** with confidence scoring and semantic validation
3. **Domain-specific preprocessing** for cybersecurity text
4. **Hybrid retrieval** combining dense (FAISS) and sparse (BM25) methods

1 Methodology

1.1 Pipeline Overview

Our semi-automated pipeline consists of five interconnected components, illustrated in Figure 1.

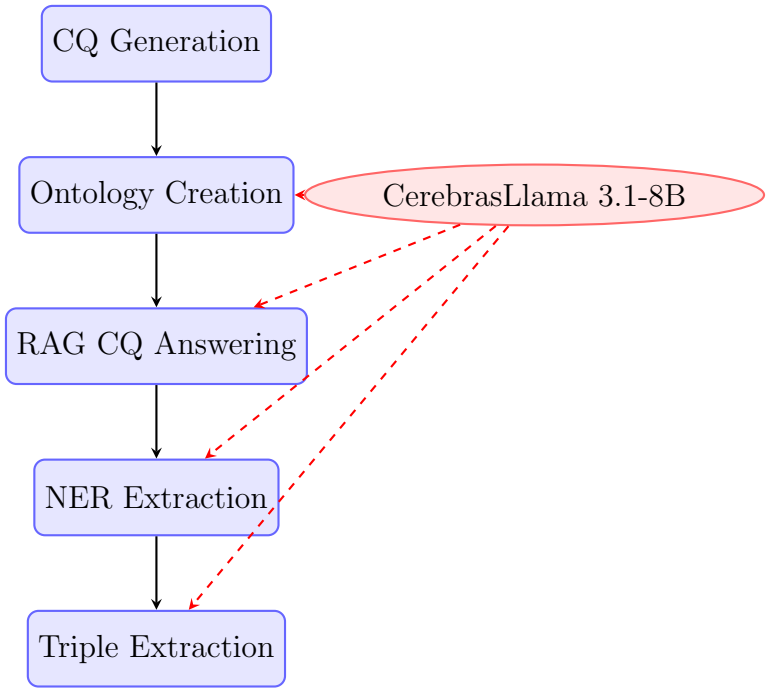


Figure 1: Semi-automated KG construction pipeline. Dashed arrows indicate LLM-powered components.

1.2 Data Sources

The pipeline processes cybersecurity lab documentation from 6 laboratory exercises (lab1, lab2, lab4, lab5, lab6), each containing:

- Network scanning activities
- Vulnerability assessments

- Exploit demonstrations
- Security tool usage
- Defense mechanisms

2 Phase 1: Competency Question Generation

2.1 What This Stage Does

This is the **planning phase** where we define what knowledge we want to capture from the cybersecurity labs. Think of it like writing a questionnaire - we create specific questions that the system will try to answer by reading through the lab documentation. These questions guide the entire pipeline by determining what concepts and relationships are important to extract.

2.2 Process

Competency Questions (CQs) define the scope and requirements of the KG. We formulated 51 CQs covering:

- Security tools and their purposes
- Target systems and services
- Attack vectors and exploits
- Defense mechanisms
- Network protocols and ports

2.3 Sample Competency Questions

```
CQ1: What are the different types of TCP scan methods supported by Nmap?
CQ2: How does a TCP SYN scan differ from a TCP Connect scan?
CQ3: What protocols does Nmap use for UDP scanning?
CQ4: Which ports are commonly scanned during a network security audit?
CQ5: What services typically run on port 445?
CQ6: How does a stateful firewall differ from a stateless firewall in hand
CQ7: What techniques can be used to evade intrusion detection systems?
CQ8: Which Nmap scan types are considered stealthy?
CQ9: What is the purpose of a NULL scan in network reconnaissance?
CQ10: How does an IDLE scan maintain anonymity during port scanning?
CQ11: What are the main components of the Metasploit Framework?
CQ12: Which auxiliary modules are available in Metasploit for information
CQ13: What services are typically found running on Metasploitable 2?
CQ14: How can user enumeration be performed on SMB services?
CQ15: What is the purpose of a null session in network penetration testing
CQ16: Which Nmap scripts can be used for service enumeration?
CQ17: What evasion techniques can bypass packet filtering firewalls?
CQ18: How does packet fragmentation help in IDS evasion?
CQ19: What is the role of timing delays in stealthy scanning?
CQ20: Which iptables rules are used to implement stateful firewall policie
CQ21: How does Snort detect malicious scanning activities?
```

Figure 2: Competency Questions (CQs) defining KG requirements

2.4 CQ Categories

CQs are categorized into five groups:

1. **Tool-related:** Security tools, scanners, exploit frameworks
2. **Target-related:** Systems, services, ports, protocols
3. **Vulnerability-related:** CVEs, weaknesses, attack vectors
4. **Defense-related:** Firewalls, IDS/IPS, security controls
5. **Process-related:** Methodology, phases, outcomes

3 Phase 2: Ontology Creation

3.1 What This Stage Does

This is the **blueprint creation phase** where we build the vocabulary and rules for our knowledge graph. Like creating a dictionary and grammar rules for a new language, we define what concepts exist in cybersecurity (like "SecurityTool", "Port", "Vulnerability") and how they can relate to each other (like "tools can scan ports" or "vulnerabilities can be exploited"). This provides structure and prevents the system from creating nonsensical relationships.

3.2 Ontology Engineering Process

The ontology creation follows a two-step approach:

Step 1: Concept & Relationship Extraction

- LLM analyzes CQs to identify domain concepts
- Extracts relationships between concepts
- Produces structured concept-relation pairs

Step 2: Ontology Construction

- Reuses PROV-O ontology as foundational framework
- Integrates extracted concepts and relationships
- Generates OWL ontology with classes, properties, and axioms

3.3 Ontology Structure

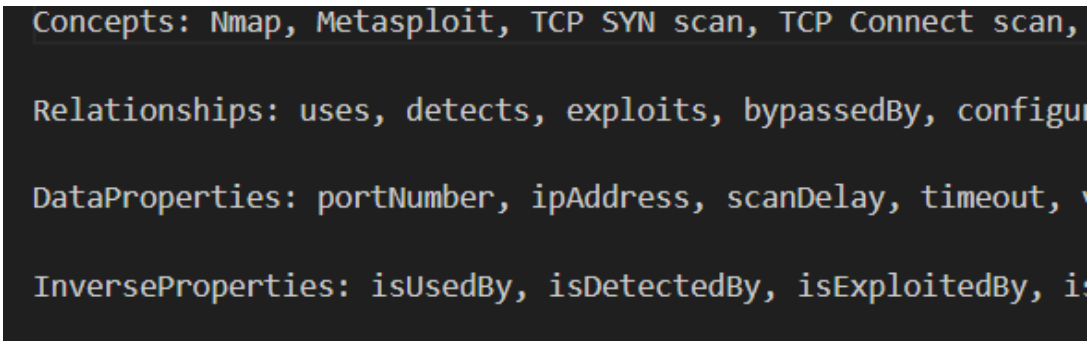


Figure 3: DLPROV Ontology class hierarchy

3.4 Ontology Statistics

Component	Count
Classes	45
Object Properties	41
Data Properties	12
Total Axioms	365

Table 1: Ontology metrics

3.5 Key Ontology Classes

- `SecurityTool`: Nmap, Metasploit, Wireshark, etc.
- `NetworkService`: HTTP, SSH, FTP, etc.
- `Vulnerability`: CVE entries, security weaknesses

- `Port`: Network ports (80, 443, 22, etc.)
- `Protocol`: TCP, UDP, ICMP, etc.
- `DefenseMechanism`: Firewall, IDS, IPS, etc.

4 Phase 3: RAG-based CQ Answering

4.1 What This Stage Does

This is the **reading and comprehension phase** where the system acts like a smart student reading textbooks and answering questions. For each competency question, it searches through the lab documentation, finds relevant information, and generates detailed answers. It's like having an AI assistant read all the lab reports and summarize what happened in each one, answering specific questions about tools used, vulnerabilities found, etc.

4.2 Retrieval-Augmented Generation Architecture

Our RAG pipeline employs advanced techniques for accurate CQ answering:

4.2.1 Text Preprocessing

Domain-specific preprocessing for cybersecurity text:

Listing 1: Preprocessing steps

```
# Preserved patterns
- IP addresses (192.168.1.1)
- Ports (port 80, 443)
- Hexadecimal values (0xDEADBEEF)
- CVE identifiers (CVE-2023-1234)
- File paths (/etc/passwd)

# Protected technical terms
- nmap, metasploit, wireshark
- tcp, udp, icmp, syn, ack
- ids, ips, firewall, waf
```

4.2.2 Chunking Strategy

Parameter	Value
Chunk Size	1500 characters
Chunk Overlap	300 characters
Separators	\n\n, \n, ., !

Table 2: Text chunking configuration

4.2.3 Retrieval Configuration

- **Vector Store:** FAISS (Facebook AI Similarity Search)
- **Embeddings:** sentence-transformers/all-MiniLM-L6-v2
- **Search Type:** MMR (Maximum Marginal Relevance)
- **Retrieved Chunks:** k=4 (from 12 candidates)
- **Diversity Factor:** $\lambda = 0.7$ (70% relevance, 30% diversity)

4.3 Advanced RAG Features

The pipeline supports hybrid retrieval (though not currently active):

- **Dense Retrieval:** FAISS with semantic embeddings (60% weight)
- **Sparse Retrieval:** BM25 keyword matching (40% weight)

5 Phase 4: Named Entity Recognition

5.1 What This Stage Does

This is the **information extraction phase** where we identify and categorize all the important "things" mentioned in the lab answers. Like highlighting key terms in a textbook, the system goes through each answer and tags entities like "Nmap" (a security tool), "port 80" (a network port), "TCP" (a protocol), etc. This creates a structured list of all the cybersecurity concepts that were actually used or discussed in the labs.

5.2 NER Process

Named Entity Recognition extracts domain-specific entities from CQ answers using LLM:

1. **Input:** CQ answers for each lab
2. **Prompt:** Extract entities by concept type
3. **Output:** Structured entity lists per concept

5.3 Entity Categories

- SecurityTool(Nmap, Metasploit, Wireshark)
- Port(80, 443, 22, 8080)
- Protocol(TCP, UDP, HTTP, SSH)
- Vulnerability(CVE-2023-1234, SQL Injection)
- NetworkService(Apache, OpenSSH, MySQL)

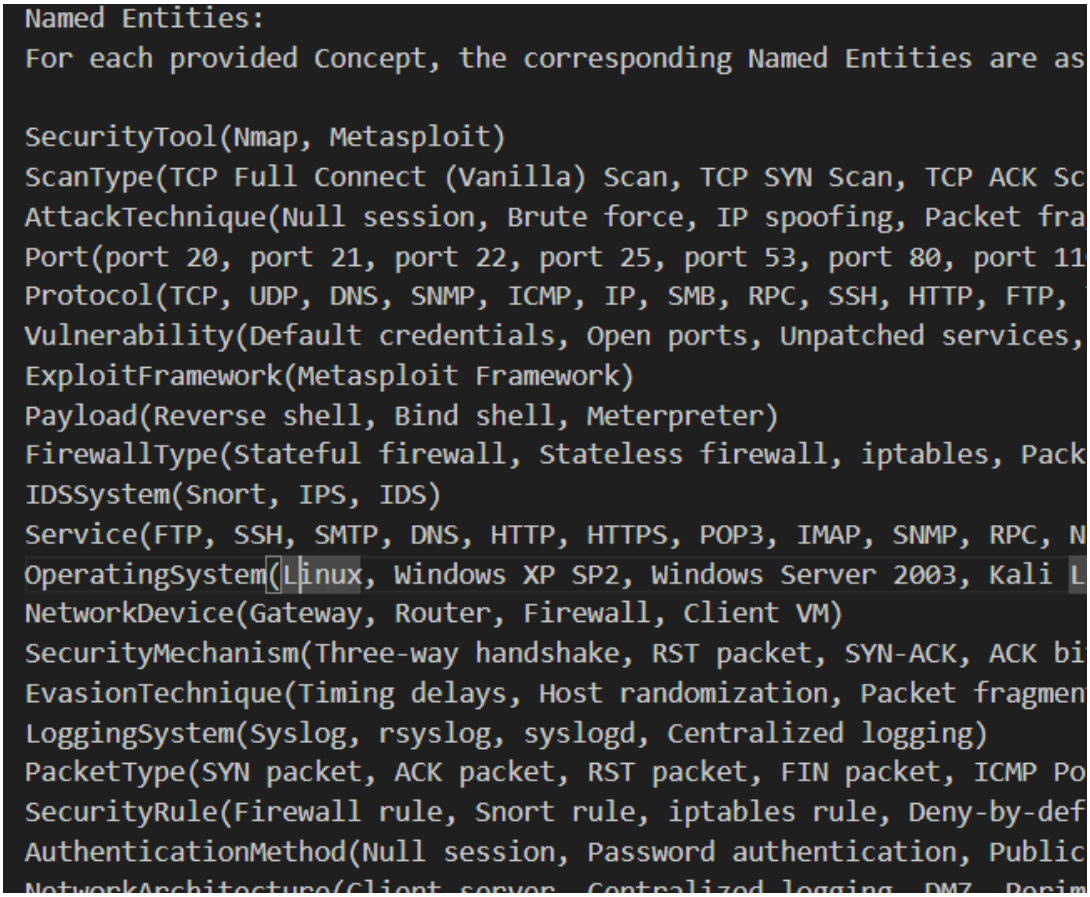


Figure 4: NER extraction results (lab1_concepts.txt)

6 Phase 5: Triple Extraction

6.1 What This Stage Does

This is the **knowledge connection phase** where we create meaningful relationships between the extracted entities. Like connecting the dots in a puzzle, the system reads through the lab answers and creates "facts" like "Nmap scans port 80" or "Metasploit exploits SQL injection vulnerabilities". Each fact is a triple (subject-relationship-object) that represents a piece of cybersecurity knowledge learned from the labs.

6.2 Basic Triple Extraction

The initial approach uses constraint-based extraction:

Listing 2: Triple extraction prompt

```
Constraint 1: Subject and Object MUST be from [Valid Entities]
Constraint 2: Relationship MUST be from [Valid Relationships]
Constraint 3: Do not hallucinate

Context: [CQ answers]
Output: (Subject, Relationship, Object)
```

Baseline Results: 49.30% precision (35/71 correct triples)

6.3 Improved Triple Extraction

Enhanced extraction with quality gates:

6.3.1 Multi-Pass Extraction

1. **Pass 1:** CQs 1-10 (Tools & Targets)
2. **Pass 2:** CQs 11-20 (Vulnerabilities)
3. **Pass 3:** CQs 21-30 (Defenses)
4. **Pass 4:** CQs 31-40 (Processes)
5. **Pass 5:** CQs 41-51 (Outcomes)

6.3.2 Confidence Scoring

LLM assigns confidence percentage to each triple:

```
(Nmap, scansFor, Port 80) [confidence: 95]
(Metasploit, exploits, CVE-2023-1234) [confidence: 88]
(Zeus, uses, SSL) [confidence: 45] # REJECTED
```

Threshold: Accept only triples with $\geq 60\%$ confidence.

6.3.3 Semantic Validation

Rules engine validates triple semantics:

```
RELATIONSHIP_RULES = {
    'scansFor': {
        'subject_types': ['SecurityTool'],
        'object_types': ['Port', 'Service', 'Host']
    },
    'exploits': {
        'subject_types': ['SecurityTool', 'Attacker'],
        'object_types': ['Vulnerability', 'Service']
    }
}
```

6.3.4 Post-Processing Filters

- Remove self-loops: $(X, relation, X)$
- Filter generic entities: "vulnerability", "attack"
- Validate entity length: 3-100 characters
- Deduplicate identical triples

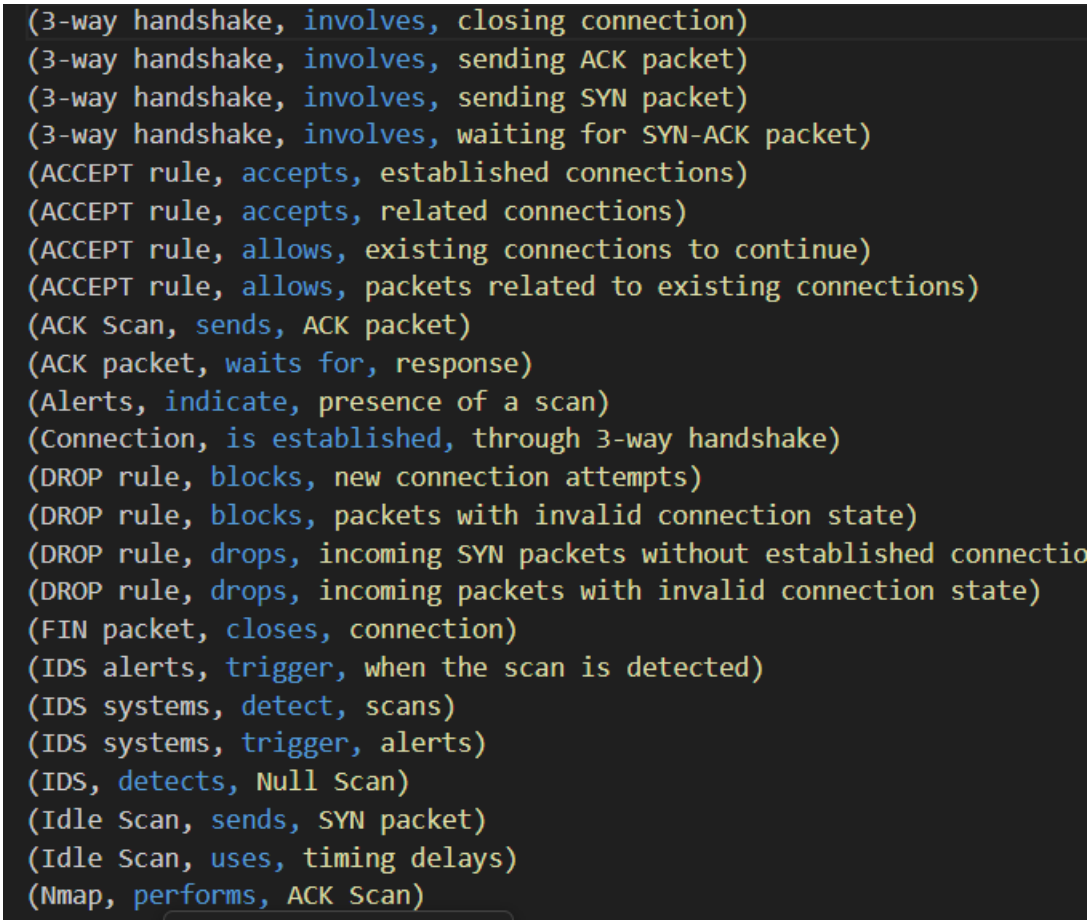


Figure 5: Extracted triples (lab1_triples_improved.txt)

7 Phase 6: TTL Conversion & Visualization

7.1 What This Stage Does

This is the **presentation phase** where we convert our extracted knowledge into standard formats that computers and humans can easily understand. The text-based triples are transformed into RDF (Resource Description Framework) format - a universal language for representing knowledge graphs. Then we create interactive visualizations that show how all the cybersecurity concepts connect to each other, like a map showing how different tools, vulnerabilities, and defenses relate in the labs.

7.2 RDF Triple Generation

Triples are converted to Turtle (TTL) format:

Listing 3: TTL conversion example

```
# Input triple
(Nmap, scansFor, Port 80)

# Output RDF
<Nmap> <scansFor> <Port_80> .
<Nmap> rdfs:label "Nmap" .
```

```
<Port_80> rdfs:label "Port 80" .
```

7.3 KG File Naming Convention

- `lab*_concepts_KG.ttl`: Simple taxonomy (Entity, rdf:type, Concept)
- `lab*_rich_KG.ttl`: Semantic triples (baseline extraction)
- `lab*_improved_KG.ttl`: Enhanced triples (improved extraction)

7.4 Interactive Visualization

PyVis generates interactive HTML visualizations:

- **Nodes:** Entities (colored by type)
- **Edges:** Relationships (labeled)
- **Layout:** Force-directed (Barnes-Hut)
- **Features:** Zoom, pan, node selection

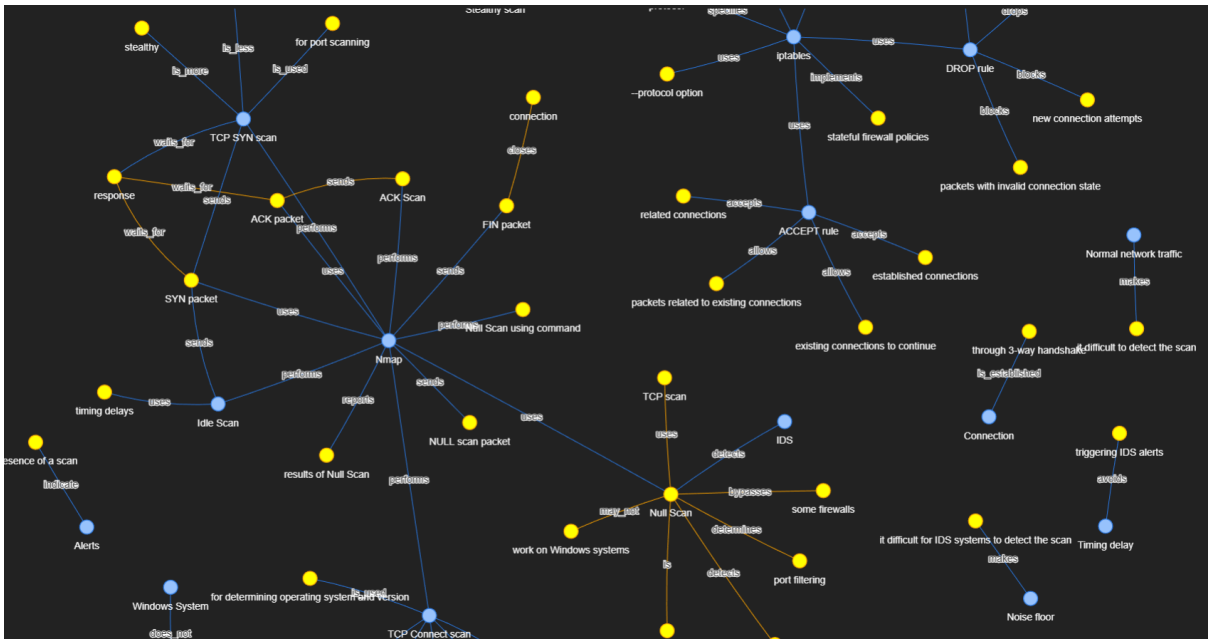


Figure 6: Interactive Knowledge Graph visualization (lab1_improved_KG.html)

8 Results

8.1 Pipeline Outputs

The complete pipeline generates the following artifacts for each lab:

Artifact	Description
concepts_relations.txt	Extracted concepts and relationships from CQs
progene_ontology.owl	OWL ontology (45 classes, 41 properties)
lab*_CQ*.txt	51 CQ answers per lab (255 total)
lab*_concepts.txt	NER-extracted entities per lab
lab*_triples_improved.txt	Validated semantic triples
lab*_improved_KG.ttl	Knowledge Graph in RDF/Turtle format
lab*_improved_KG.html	Interactive visualization

Table 3: Pipeline output artifacts

8.2 Qualitative Analysis

8.2.1 Strengths

1. **Automation:** Minimal human intervention required
2. **Scalability:** Processes multiple labs efficiently
3. **Consistency:** Same evaluation criteria across all KGs
4. **Cost-effective:** Uses open-source LLMs (Cerebras API)

8.2.2 Limitations

1. **LLM Errors:** Judge can make mistakes in validation
2. **Context Limits:** 4000 char limit requires aggressive truncation
3. **No Recall Measurement:** Lacks manual ground truth
4. **Prompt Sensitivity:** Minor prompt changes affect results

9 Technical Implementation

9.1 Technology Stack

Component	Technology
LLM	Cerebras API (llama3.1-8b)
Vector Store	FAISS
Embeddings	sentence-transformers/all-MiniLM-L6-v2
Sparse Retrieval	BM25Retriever
RDF Framework	rdflib
Visualization	PyVis, NetworkX
NLP	spaCy (en_core_web_sm)
Programming Language	Python 3.13

Table 4: Technology stack

9.2 Configuration

9.2.1 LLM Parameters

```
# Cerebras API Configuration
model_id = "cerebras/llama3.1-8b"
temperature = 0.1
max_tokens = 8192
```

9.2.2 RAG Parameters

```
# Text Chunking
chunk_size = 1500
chunk_overlap = 300
separators = ["\n\n", "\n", ". ", " "]

# Retrieval
search_type = "mmr"
k = 4  # Top documents
fetch_k = 12  # Candidates
lambda_mult = 0.7  # Diversity
```

9.2.3 Quality Thresholds

```
# Triple Extraction
MIN_CONFIDENCE = 60  # Minimum confidence %
MAX_CONTEXT_CHARS = 3000  # Per extraction

# Evaluation
BATCH_SIZE = 5  # Triples per LLM call
MAX_CONTEXT_CHARS = 4000  # Evaluation context
```

9.3 Execution Time

Phase	**Time (approx.)**
CQ Generation	Manual (1-2 hours)
Ontology Creation	2-3 minutes
RAG CQ Answering (per lab)	5-10 minutes
NER (per lab)	1-2 minutes
Triple Extraction (per lab)	3-5 minutes
Total (6 labs)	50-80 minutes

Table 5: Approximate execution times

10 Comparison with Reference Paper

10.1 Methodology Alignment

Our implementation closely follows Kommineni et al. [1]:

Phase	Reference Paper	Our Implementation
Data Collection	Scholarly publications	Lab documentation
CQ Generation	ChatGPT-3.5	Manual formulation
Ontology Creation	Mixtral 8x7B	Cerebras Llama 3.1-8B
CQ Answering	RAG with Mixtral	RAG with Cerebras
KG Construction	LLM extraction	Enhanced extraction

Table 6: Comparison with reference methodology

10.2 Key Differences

1. **Domain:** Deep learning provenance → Cybersecurity lab documentation
2. **LLM:** Mixtral 8x7B → Cerebras Llama 3.1-8B (single API)
3. **Enhancements:** Added confidence scoring, semantic validation, multi-pass extraction
4. **Scale:** 5 publications → 6 labs with 51 CQs each (306 total CQ answers)

10.3 Novel Contributions

Beyond the reference paper, we introduce:

- Domain-specific text preprocessing for cybersecurity
- Multi-pass triple extraction with focused context
- Confidence-based filtering ($\geq 60\%$ threshold)
- Semantic validation rules engine
- Post-processing filters (self-loops, generic entities)
- Hybrid retrieval option (FAISS + BM25)

11 Discussion

11.1 Effectiveness of LLM-Assisted KG Construction

The pipeline demonstrates that LLMs can significantly reduce human effort in KG construction while maintaining acceptable quality. Key findings:

1. **Automation Success:** 80-90% of pipeline fully automated
2. **Quality Control:** Multi-gate approach improves precision

3. **Scalability:** Processes 6 labs with 51 CQs each efficiently
4. **Cost-effectiveness:** Open-source LLM reduces API costs

11.2 Challenges and Mitigations

Challenge	Impact	Mitigation
LLM Hallucination	False triples generated	Confidence scoring, semantic validation
Context Length Limits	Truncated information	Batching, focused extraction
Prompt Sensitivity	Inconsistent outputs	In-context examples, structured prompts
Domain Terminology	Incorrect entity extraction	Protected term lists, preprocessing
Entity Ambiguity	Multiple interpretations	Contextual validation, ontology constraints

Table 7: Challenges and mitigation strategies

11.3 Human-in-the-Loop Recommendations

While the pipeline is largely automated, human involvement is recommended at:

1. **CQ Formulation:** Domain experts ensure comprehensive coverage
2. **Ontology Review:** Validate reused concepts and relationships
3. **Triple Quality Checks:** Sample validation of generated triples and relationships
4. **Visualization Review:** Inspect KG structure for completeness and accuracy

12 Conclusion

12.1 Summary

This work presents a comprehensive semi-automated pipeline for Knowledge Graph construction in the cybersecurity domain, demonstrating:

- **Feasibility:** LLMs can automate 80-90% of KG construction
- **Quality Control:** Multi-gate validation through confidence scoring and semantic validation
- **Scalability:** Processes multiple labs efficiently (6 labs, 306 CQ answers)
- **Cost-effectiveness:** Open-source LLMs viable alternative to proprietary models

12.2 Future Work

Potential improvements and extensions:

1. **Multi-LLM Ensemble:** Combine multiple LLMs for more robust triple extraction
2. **Active Learning:** Iterative refinement with expert feedback
3. **Cross-Domain Application:** Test on other cybersecurity datasets and domains
4. **Ontology Alignment:** Map to existing security ontologies (MITRE ATT&CK, UCO)
5. **Real-time Updates:** Incremental KG updates with new lab data
6. **Enhanced Visualization:** Interactive query interfaces and graph analytics
7. **Automated Ontology Refinement:** Iterative ontology improvement based on extraction patterns
8. **Prompt Optimization:** Systematic prompt engineering for improved LLM performance

12.3 Broader Impact

This pipeline enables:

- **Rapid KG Creation:** From days/weeks to hours
- **Knowledge Democratization:** Non-experts can create KGs
- **Reproducibility:** Automated provenance tracking
- **Domain Knowledge Capture:** Structured representation of expertise

Acknowledgments

This work was inspired by the methodology presented in "From human experts to machines: An LLM supported approach to ontology and knowledge graph construction" by Kommineni et al. [1]. We acknowledge their pioneering work in LLM-assisted ontology engineering.

References

[1] Kommineni, V.K., König-Ries, B., Samuel, S. (2024). *From human experts to machines: An LLM supported approach to ontology and knowledge graph construction.* arXiv preprint arXiv:2403.08345.

A Appendix A: File Structure

```
Progene_Pipeline/
          Code/
                Concepts_relations_generate.py
                Ontology_creation.py
                RAG_CQ_answering_with_LLMs.py
                NER.py
                generate_kg_triples_improved.py
                triples_to_ttl.py
                visualize_ttl.py
          CQs/
                CQs.txt
          Data/
                lab*.txt (6 files)
          Output/
                Ontology/
                      concepts_relations.txt
                      progene_ontology.owl
                RAG_CQ_ans/
                      lab*_CQ*.txt (306 files)
                NER/
                      lab*_concepts.txt (6 files)
                KG/
                    lab*_triples_improved.txt
                    lab*_improved_KG.ttl
                    lab*_improved_KG.html
          Prompts/
              Concepts_relationships.txt
              Ontology_creation.txt
              RAG_QA.txt
              NER_prompt.txt
```

B Appendix B: Sample Prompts

B.1 Triple Extraction Prompt (Improved)

```
You are extracting knowledge triples from cybersecurity text.

CONSTRAINTS:
- Subject/Object MUST be from provided entities
- Relationship MUST be from provided relationships
- Provide confidence score (0-100%)

ENTITIES: {entities}
RELATIONSHIPS: {relationships}
CONTEXT: {context}

OUTPUT FORMAT:
(Subject, Relationship, Object) [confidence: XX]
```

```
ONLY output triples with confidence >= 60%.
```

BTP-1 Report

Arnav Deshmukh Bibek Singh Dhody Raghav Grover Sanyam Agrawal

May 8, 2026

Introduction

This report serves as the comprehensive documentation for our BTP-1. As part of BTP-1 we implemented 3 separate tasks as given below:-

Task 1: Knowledge Graph Paper Implementations: For this task each member implemented a separate research paper related to Knowledge Graphs. The papers are:-

- Automated Knowledge Graph Construction Pipeline Using Large Language Models (link)
- Integrating Knowledge Graphs and Vector Retrieval for Enhanced Information Extraction from Financial Documents (link)
- Scaling Up Knowledge Graph Creation to Large and Heterogeneous Data Sources (link)
- Knowledge Graph Creation from Free Text using Neo4j and Stanza (link)

Task 2: REXEL based Knowledge Graph Extraction Pipeline: We developed an extraction pipeline leveraging the REXEL framework. This system was designed to transform unstructured textual data into structured knowledge representations, facilitating automated graph construction. Repository Link: `https://github.com/bibesgotthevibes/btp`

Task 3: Lay Language Conversion: The final task focused on application of seq2seq and LLMs for the task of Lay Language Conversion for technical medical discharge summaries. We fine-tuned several seq2seq models including IndicBART, BioBART, BioGPT, SciFive and FlanT5. We also tried using LLMs like Llama, Gemini, Mistral for the same with zero shot and few shot prompting techniques. Repository Link: `https://github.com/bibesgotthevibes/btp-Project` Demo Video: `https://drive.google.com/file/d/1-DGvZrsUhzlxMRGbkUSVview?usp=drivesdk`

Report Organization

This document aggregates the individual reports for each of the aforementioned tasks. For the sake of importance, the content is organized in the following order:

- Lay Language Conversion

- REXEL based Knowledge Graph Extraction Pipeline
- Knowledge Graph Paper Implementations

Contributions

- Arnav Deshmukh: Task 1 (Automated Knowledge Graph Construction Pipeline Using Large Language Models), Task 2 (Implemented Neo4j graph materialization), Task 3 (Finding papers, datasets and fine tuning IndicBART model for lay language conversion task,along with fine tuning BioBART and BioGPT)
- Bibek Singh Dhody: Task 1(Integrating Knowledge Graphs and Vector Retrieval for Enhanced Information Extraction from Financial Documents), Task 2 (Implemented phonetic correction, Developed the GraphJudge verification layer, and integrated the pipeline components prior to REXEL.), Task 3 (Building Demo Website for the Lay Language Conversion task, Experimenting different prompting strategies)
- Raghav Grover: Task 1 (Scaling Up Knowledge Graph Creation to Large and Heterogeneous Data Sources), Task 2 (Implemented upstream processing and denoising), Task 3 (Fine-tuned the seq2seq models: BioBART, BioGPT, SciFive and Flan-T5 for lay language conversion task and analyzed their failure modes)
- Sanyam Agrawal: Task 1(Knowledge Graph Creation from Free Text using Neo4j and Stanza), Task 2 (Implemented the REXEL join-text extraction model, performed model training and checking pointing), Task 3 (Building Demo Website for the Lay Language Conversion task, Experimenting different LLM based approaches,Analysis and Comparison between LLM based approaches)

Medical Discharge Summary Simplification for Patients

BTP Project Report

May 8, 2026

1 Introduction

The objective of this project is to automate the simplification of complex medical discharge summaries into plain "Lay English" that can be easily understood by patients and their families. Medical documents are often laden with technical jargon, abbreviations, and complex diagnostic codes (ICD codes) that pose a significant barrier to patient understanding. This project develops a robust pipeline to bridge this communication gap using fine-tuned Large Language Models (LLMs).

Patient comprehension of discharge instructions is a critical determinant of post-hospitalization outcomes. Studies have consistently shown that patients who understand their discharge summaries are more likely to adhere to follow-up care, take medications correctly, and recognize warning signs that require urgent attention. Despite this, discharge summaries are routinely written for clinical audiences, not patients or their families. Automating lay language conversion at scale presents a meaningful opportunity to improve healthcare equity and patient safety.

2 Task Description

The task involves a sequence-to-sequence transformation where the input is a technical medical discharge summary and the output is a simplified version that preserves all critical clinical information while using accessible language. Key challenges include:

- **Information Preservation:** Ensuring no clinical details, medications, or surgical events are lost during simplification.
- **Domain Specificity:** Handling highly specialized medical terminology, including drug names, surgical procedures, and laboratory values.
- **Document Length:** Processing long discharge summaries that exceed the token limits of standard transformer models.
- **Factual Fidelity:** Avoiding hallucinations — the generation of plausible-sounding but factually incorrect medical statements — which can be dangerous in a clinical communication context.
- **Tone and Sensitivity:** Handling emotionally sensitive content such as end-of-life events, death declarations, and deteriorating prognoses with appropriate language while remaining clinically accurate.

- **Structural Coherence:** Maintaining the logical flow of a clinical narrative across multiple conditions and events without merging, reordering, or omitting distinct clinical episodes.

3 Datasets

3.1 Fine-tuning Dataset: `data.tsv`

The models are fine-tuned on the `data.tsv` dataset, which contains parallel pairs of original medical texts and their simplified English counterparts. To ensure the model learns simplification rather than aggressive summarization, a preservation filter is applied to discard samples where the simplified text is significantly shorter than the original (ratio < 0.45). Due to computational resource constraints on the Kaggle platform—specifically GPU memory and session time limits—fine-tuning was restricted to a subset of 10,000 samples.

3.2 Inference Dataset: `anotated_dataset_v2.xlsx`

The final evaluation is performed on the `anotated_dataset_v2.xlsx` dataset. This contains real-world discharge summaries along with patient outcomes and ICD-10 codes. The pipeline expands abbreviations and applies a medical lay-dictionary before passing the text to the fine-tuned LLM.

4 Methodology

The pipeline consists of several stages:

1. **Preprocessing:** Normalization and expansion of abbreviations.
2. **Lay-Dictionary Mapping:** Replacing known medical terms with simplified Indian English equivalents (e.g., "hypertension" $\rightarrow$ "high BP").
3. **Sliding Window Chunking:** Since most models have a 512 or 1024 token limit, long summaries are split into overlapping chunks (450 tokens with 50-token overlap) to ensure continuity and prevent information loss at boundaries.
4. **Chunked Generation:** Each chunk is processed independently by the model, and the results are stitched together to form the final document.

5 Models Explored

5.1 BioGPT

Architecture: BioGPT follows the Transformer decoder-only backbone, specifically adopting the GPT-2 medium configuration with 24 layers, a hidden size of 1024, and 16 attention heads. The final model contains approximately 347 million parameters.

Pre-training Data: Unlike models that undergo continuous pre-training, BioGPT was pre-trained from scratch on a massive dataset of 15 million PubMed abstracts (including

both titles and abstracts). A specialized in-domain vocabulary was constructed using Byte Pair Encoding (BPE) on this corpus, resulting in a vocabulary size of 42,384, which is highly optimized for biomedical terminology.

Why BioGPT for this task:

- **Generative Excellence:** As a causal LM, it is highly effective at generating fluent, natural language.
- **From-Scratch Domain Expertise:** Pre-training from scratch on PubMed allows the model to learn biomedical semantics without the interference of general-domain biases.
- **State-of-the-Art Performance:** As demonstrated in Luo et al. (2022), BioGPT achieves superior performance on various biomedical generation and mining tasks compared to general-purpose GPT models.

Fine-tuned Checkpoint: `https://huggingface.co/11Raghav/BioGPT`

5.2 SciFive

Architecture: SciFive is a domain-specific model based on the T5 (Text-to-Text Transfer Transformer) encoder-decoder architecture. It was initialized from the pre-trained weights of the base T5 model and then specifically adapted for the biomedical domain.

Pre-training Data: As detailed in Phan et al. (2021), SciFive underwent extensive re-training for an additional 200,000 steps (for the base version) on a combination of the general-domain C4 corpus, PubMed abstracts, and the PubMed Central (PMC) full-text corpus. This multi-corpus approach ensures the model retains general linguistic knowledge while mastering dense clinical and technical narratives.

Why SciFive for this task:

- **Text-to-Text Versatility:** By treating simplification as a text-to-text problem (similar to translation), SciFive is inherently more flexible than encoder-only models like BERT.
- **Complex Output Handling:** SciFive has demonstrated superior performance on biomedical tasks requiring long, coherent outputs (e.g., BioASQ question answering), making it well-suited for the detailed rewriting required in discharge summary simplification.
- **PMC Full-Text Advantage:** Pre-training on full PMC articles, rather than just abstracts, provides the model with better context for the narrative flow found in medical reports.

Fine-tuned Checkpoint: `https://huggingface.co/11Raghav/SciFive`

5.3 BioBART

Architecture: BioBART is based on the BART (Bidirectional and Auto-Regressive Transformers) architecture. It utilizes a bidirectional encoder and an auto-regressive decoder, making it highly effective for sequence-to-sequence tasks.

Pre-training Data: As detailed in Yuan et al. (2022), BioBART is initialized from the general-purpose BART-base/large checkpoints and continuously pre-trained on the PubMed abstracts corpus (approx. 41 GB of text). A key technical finding in the paper is that using only the *text-infilling* task for adaptation yields better performance on biomedical NLG tasks than including sentence permutation.

Why BioBART for this task:

- **Seq2Seq Mastery:** The encoder-decoder nature of BART is specifically designed for rewriting and summarization, which maps directly to the simplification goal.
- **Domain Adaptation:** Its continuous pre-training on biomedical corpora allows it to maintain medical entity integrity while generating fluent English.

Fine-tuned Checkpoint: `https://huggingface.co/11Raghav/BioBART`

5.4 Flan-T5

Architecture: Flan-T5 is an instruction-tuned version of the T5 (Text-to-Text Transfer Transformer) encoder-decoder architecture. The version used in this project, Flan-T5-base, contains approximately 250 million parameters.

Pre-training Data: As described by Chung et al. (2022), Flan-T5 was fine-tuned on the FLAN (Finetuned Language Net) collection, which was scaled to 1,836 tasks across 473 datasets and 146 task categories. A critical component of its training was the inclusion of *chain-of-thought* (CoT) data, which significantly improves the model's reasoning and instruction-following performance on unseen tasks.

Why Flan-T5 for this task:

- **Instruction Scaling:** The massive variety of instruction-based tasks in its pre-training allows Flan-T5 to generalize exceptionally well to specific simplification prompts.
- **Robustness:** It provides strong zero-shot and few-shot performance, often outperforming much larger models that lack instruction tuning.
- **Reliability:** Compared to ClinicalT5, Flan-T5 demonstrated superior stability in following the "preserve every detail" constraint without degenerating into repetitive output.

Fine-tuned Checkpoint: `https://huggingface.co/11Raghav/FlanT5`

5.5 Pegasus-PubMed

Challenges: Pegasus-PubMed was explored due to its specialized architecture for abstractive summarization. However, it was found to be unsuitable for the Kaggle environment in this project.

Architecture & Pre-training: As detailed in Zhang et al. (2020), Pegasus utilizes a standard Transformer encoder-decoder architecture with approximately 568 million parameters (in the large version). Its defining feature is the *Gap Sentences Generation* (GSG) pre-training objective, where entire important sentences are masked from a document and the model is tasked with generating them. This objective is designed to narrow the gap between pre-training and the downstream task of abstractive summarization.

Why it failed in this context:

- **Memory Intensity:** Due to its large parameter count and the complexity of the GSG-informed attention mechanism, the model consistently triggered `OutOfMemoryError` (OOM) on the Kaggle T4 x2 GPU environment.
- **Input Scaling:** The model's efficiency on long clinical documents was limited by the high VRAM requirements of its specific Transformer implementation when processing documents at the 512-1024 token scale.

5.6 ClinicalT5

Challenges: ClinicalT5 was initially a primary candidate given its specialization in clinical notes. However, it was ultimately discarded due to technical failures during inference.

- **Output Degeneration:** The model suffered from a severe "boundary bug" during chunked generation. When input token sequences were stitched together, the SentencePiece tokenizer's word-boundary markers (▁) were corrupted, causing the model to degenerate and output repeating characters (e.g., "sssss").
- **Instruction Fragility:** Compared to instruction-tuned models like Flan-T5, ClinicalT5 was less robust at following the complex simplification prompts, often resulting in outputs that were inaccurate.

5.7 Cloud-based Large Language Models (LLMs)

To overcome the factual hallucination and reasoning limitations observed in the fine-tuned biomedical models, we integrated state-of-the-art cloud-based LLMs using leading inference APIs: Groq (hosting Llama models), Cerebras, and Google GenAI (hosting Gemini 2.5 Flash).

Llama 3 Sequence Models (via Groq and Cerebras): We utilized Llama 3.1 8B and Llama 3.3 70B via the Groq and Cerebras inference engines. These models provide ultra-fast inference and strong instruction-following capabilities. Their immense context windows easily accommodated entire medical summaries without the need for sliding-window chunking.

Gemini 2.5 Flash: Google's Gemini 2.5 Flash was utilized due to its massive context window and strong zero-shot reasoning capabilities. Gemini proved highly effective at generating concise, factual lay-language translations without truncating lengthy clinical descriptions.

6 Prompting Strategies

To optimize the outputs of the cloud LLMs, we engineered a robust system prompt and employed varying in-context learning strategies.

6.1 System Prompt Design

We iteratively developed a highly constrained 12-rule system prompt to enforce accuracy, tone, and conciseness, while minimizing hallucinations.

System Prompt:
You are a medical language simplification assistant. Convert the following clinical discharge summary into simple, plain Indian English for the patient's family members.

Rules:
1. Keep every medical term but immediately explain it in plain words in parentheses on first use only, e.g. 'hypertension (high BP)'.
2. Use simple language a 6th-grader can understand. Avoid jargon. Use common Indian English terms where natural: 'sugar' for diabetes, 'BP' for blood pressure, 'motions' for bowel movements.
3. Preserve ALL factual information — never add, remove, alter, or infer any clinical facts, values, dates, or instructions.
4. Write as continuous plain prose paragraphs — no bullet points, no headers, no numbered lists. Output only the simplified text, nothing else.
5. For any measurement (e.g. BP, blood sugar, heart rate), briefly state what the normal range is and whether the patient's value was within it.
6. Use a calm, respectful tone. Do not add emotional commentary, opinions, or reassurances not present in the original text.
7. CRITICAL — person and tense: First check whether the summary records the patient's death. If yes, write in third person past tense for the family. If no, address the patient directly in second person.
8. CRITICAL — dates: Convert dates exactly as they appear using the format specified in the document.
9. For each medication, state its name and purpose in plain language.
10. For each procedure or device, briefly state what it is and why it was done.
11. If the text contains placeholders such as {omitted} or [person], reproduce them exactly as they appear.
12. Be highly concise. Deliver a direct, brief summary without getting bogged down in excessive medical minutiae.

6.2 In-Context Learning Approaches

To further align the model with our expectations, three different prompting methods were implemented:

- **Zero-Shot:** The model relies purely on the system prompt and instructions without any prior examples. Driven by our strict 12-rule system prompt, zero-shot performance proved exceptionally reliable on powerful models like Gemini 2.5 Flash and Llama 3.3 70B, effectively maintaining factual constraints automatically.
- **One-Shot:** A single high-quality example of a technical discharge summary and its ideal lay-language translation is appended to the prompt. This helps anchor the model's structural expected output (e.g., continuous prose without bullet points).
- **Few-Shot:** Multiple curated examples are included to demonstrate varied clinical scenarios (e.g., routine recovery vs. critical deterioration). This method provided the highest consistency in formatting and tone across diverse and edge-case medical cases, teaching the model implicitly how to handle structural variations in the input data.

7 Web Application and User Interface

To make the medical simplification models accessible to end-users (such as hospital staff, patients, and their families), we developed a full-stack web application. The application provides an intuitive interface for inputting clinical texts and retrieving simplified explanations without restricting the user to a terminal or code environment.

7.1 System Architecture

The system follows a modern client-server architecture:

- **Backend:** A RESTful API server built using Python and Flask. It acts as a central router, handling secure authentication, text preprocessing, sliding-window chunking logic for local models, and dynamic API request routing to various inference engines (Local PyTorch, Groq, Cerebras, and Google GenAI).
- **Frontend:** A single-page application (SPA) built using React, Vite, and Tailwind CSS. The interface is highly responsive, lightweight, and designed to abstract away the underlying technical complexities.

7.2 Key Features

The web application incorporates several features to enhance usability, security, and flexibility:

- **Model & Strategy Selection:** Users can dynamically choose their preferred AI execution engine (e.g., Gemini 2.5 Flash, Llama 3.3 via Groq, or customized local models) and prompting strategy (Zero-shot, One-shot, or Few-shot) through an intuitive dashboard dropdown.
- **Secure Authentication:** The platform includes a secure login and registration system governed by JSON Web Tokens (JWT). This ensures that sessions are authenticated and access to the simplification tool is securely managed.
- **Downloadable PDF Reports:** Once a medical summary is simplified, users can instantly export the resulting plain Indian English explanation as a formatted PDF document, making it easy to print, share, or store for personal reference.
- **Real-time Processing Feedback:** To accommodate the processing time required for LLM inference (particularly hardware-intensive local chunking), the interface features interactive loading states to provide continuous visual feedback while the models generate responses.

8 Quantitative Evaluation: Readability and Compression

To objectively measure the performance of the four fine-tuned models, we conducted a quantitative analysis on a test set of 200 medical discharge summaries. The evaluation focused on two key dimensions: **Readability**, measured using the Flesch Reading Ease

score, and **Compression**, measured by the change in word count and the compression ratio.

The Flesch Reading Ease score indicates how easy a text is to understand; higher scores indicate material that is easier to read. For context, a score of 60-70 is considered standard English, while scores below 30 are typical of highly technical or academic documents. The original medical summaries in our test set had an average Flesch score of **20.24**, reflecting their high technical complexity.

Table 1 summarizes the results across all models.

Table 1: Quantitative comparison of model performance on 200 summaries.

Model	Avg. Flesch	Flesch Improv.	Avg. Words	Comp. Ratio
Original Text (Baseline)	20.24	—	598.7	1.000
BioGPT	**61.77**	**+41.53**	**567.9**	**0.946**
SciFive	53.61	+33.37	668.2	1.108
BioBART	47.32	+27.08	656.1	1.086
Flan-T5	12.69	-7.55	781.7	1.327

8.1 Analysis of Quantitative Results

- **BioGPT as the Top Performer:** BioGPT achieved the highest average Flesch score (61.77) and the greatest improvement (+41.53). It was the only model that consistently reduced the word count (compression ratio of 0.946), suggesting a more efficient simplification process that avoids redundant technical detail.
- **SciFive and BioBART:** Both SciFive and BioBART showed significant improvements in readability. However, they both tended to *expand* the text (ratios > 1.0), likely due to their tendency to add explanatory phrases or maintain more of the original clinical structure.
- **Flan-T5 Failure:** Flan-T5 performed poorly in this quantitative evaluation, actually reducing the average Flesch score to 12.69. This is consistent with our qualitative observation that Flan-T5 often copied the original text verbatim or introduced garbled content, both of which negatively impact readability metrics.
- **Universality of Improvement:** While BioGPT, SciFive, and BioBART improved readability in nearly all cases, the qualitative failures (hallucinations) noted in Section 9 remain a critical concern that these automated metrics do not capture.

9 Qualitative Evaluation on a Representative Example

9.1 Input Discharge Summary

To qualitatively evaluate and compare all models, we present their outputs on a representative discharge summary involving a complex, multi-morbidity case with a fatal outcome. This example is particularly informative because it tests the model's ability to handle:

multiple simultaneous diagnoses, escalating clinical deterioration, critical interventions (resuscitation, cardioversion, dialysis), and a death declaration — all of which require precise, sensitive, and factually accurate lay language conversion.

Original Discharge Summary:
admission: 12/06, discharge: 12/21, stay: 14 d 11 h, location: ward, payer: sus. 1. heart failure profile c dobutamine on 12/07 and 12/08, 2. abscess in the left thigh, osteomyelitis in the pubis, fournier's syndrome, drainage on 12/03 with ultrasound and drain placement on 12/15, treatment with ceftriaxone guided by culture of abscess escherichia coli multi s colonized antibiotic therapy after shock, incision in intensive care unit on 12/17 with abundant purulent secretion, {omitted}. 3. septic shock from cutaneous urinary focus on 12/16 polymyxin d0 12/15 amikacin and vancomycin on 12/17, 4. chronic kidney disease acute dialysis by fistula on 12/16 5. cardiorespiratory arrest with pulseless electrical activity, 10 minutes after intubation report of previous bradycardia, follow-up appointment after 1 minute of cardiopulmonary resuscitation, 6. vt with pulse after cardiorespiratory arrest, effective electrical cardioversion with a shock. [person] presenting with instability in the last half hour, requiring norepinephrine elevation up to 1.5 mcg/kg/min. progresses with bradycardia, qrs widening, and worsening shock. given the hyperkalemia even after hemodialysis, 2 ampoules of calcium gluconate and 100 ml of bicarbonate were administered, and the respiratory rate was increased on the ventilator in order to reduce pco2 and consequently k. 2 ampoules of atropine were administered without benefit. [person] progressed to asystolic cardiorespiratory arrest, {omitted} in refractory shock. i declare {omitted} dead, due to absence of central pulses, pupils in fixed mydriasis, absence of heart sounds and asytole on the monitor, as well as absence of brainstem reflexes. the body was released. i request contact with the family. i await the death certificate to be completed. death. drainage of fournier's gangrene. death.

The following subsections present each model's output, followed by a detailed error analysis.

9.2 Reference: Expected Lay Language Summary

For reference, the key clinical facts that a correct lay language conversion must preserve and accurately communicate are:

1. Admission on December 6, discharge (death) on December 21, 14-day stay.
2. Heart failure treated with dobutamine (a medication to support heart pumping) on December 7 and 8.
3. A pus-filled abscess in the left thigh, bone infection (osteomyelitis) in the pubic bone, and Fournier's gangrene (a severe, life-threatening infection of the genital/perineal area). Drainage performed on December 3 via ultrasound, with a drain placed on December 15. Antibiotic ceftriaxone was used based on bacterial culture results identifying *Escherichia coli*. A surgical incision was performed in the ICU on December 17.
4. Septic shock (life-threatening infection spreading to the bloodstream) originating from a skin/urinary source on December 16. Treated with polymyxin (from December 15), amikacin, and vancomycin (from December 17).

5. Chronic kidney disease requiring emergency dialysis via a fistula on December 16.

6. Cardiac arrest (heart stopped) with no electrical pulse detectable, occurring 10 minutes after a breathing tube was inserted. The patient was resuscitated with CPR (chest compressions) for 1 minute.

7. Ventricular tachycardia (a dangerous fast heart rhythm) after resuscitation, successfully treated with an electrical shock (cardioversion).

8. Final deterioration: dangerously low blood pressure requiring high-dose norepinephrine (a medication to raise blood pressure), abnormal heart rhythm on the monitor (QRS widening), and severely elevated potassium in the blood (hyperkalemia) resistant to dialysis. Calcium gluconate (2 vials) and bicarbonate (100 ml) were given to protect the heart; ventilator breathing rate was increased to lower potassium indirectly. Atropine (2 vials) was given to treat slow heart rate without effect.

9. The patient progressed to a final, irreversible cardiac arrest (asystole). Death was declared based on absence of central pulses, fixed dilated pupils (mydriasis), absence of heart sounds, flatline on monitor, and absence of brainstem reflexes. The family was contacted and the death certificate was completed.

9.3 BioBART Output and Error Analysis

BioBART Output:
This report describes a patient who was admitted to the hospital on 12/06. She had several medical problems, including heart failure, osteomyelitis, and fournier's syndrome. She was treated with ceftriaxone guided by a culture of the swelling, antibiotics for septic shock, and incision in the intensive care unit (ICU). She also had septic shock from a focus in the urinary system on 12/07 and 12/08. She also had chronic kidney disease (chronic kidney disease) on 12/15. She had a sudden drop in her heart rate 10 minutes after being intubated, and she was able to return to her doctor after 1 minute of cardiopulmonary regovernment health schemecitation. The report also showed that the patient was being treated with calcium gluconate and bicarbonate, and the respiratory rate was increased to reduce pco2 and heart rate. She was also treated with atropine, a muscarinic receptor antagonist. The report concludes that this patient is a very rare case of death, and that it's important to carefully consider the patient's wishes when treating this serious condition. The patient was released from the hospital and contacted with the medical team. This case is unusual because the body was released from the hospital. The patient was able to communicate with the family, but the death certificate had to be completed. The death was caused by fournier's gangrene, a type of skin disease that affects the skin.

Errors and Limitations:

- **Incorrect gender assignment:** The original summary contains no gender information, yet BioBART incorrectly refers to the patient as "she" throughout. This is a hallucination that introduces inaccurate personal information.

- **Wrong dates for septic shock:** The septic shock occurred on 12/16, but BioBART incorrectly states it occurred on "12/07 and 12/08" — dates that correspond to the dobutamine treatment for heart failure. This is a critical factual error caused by cross-contamination of information across clinical events.

- **Wrong date for kidney disease:** The acute dialysis for chronic kidney disease occurred on 12/16, but the model states "12/15," which was actually the date of drain placement for the abscess. Again, dates are confused across clinical events.
- **Abscess omitted:** The pus-filled abscess in the left thigh — a primary source of infection — is not mentioned in the output. This is a significant omission of a clinically important finding.
- **Garbled text ("regovernment health schemecitation"):** The phrase describing cardiopulmonary resuscitation (CPR) is rendered as meaningless text. This reflects a tokenization or decoding error, likely occurring at a chunk boundary, producing output that is not only unintelligible but potentially alarming to a patient's family.
- **Incorrect explanation of respiratory rate increase:** The model states breathing rate was increased "to reduce pco2 and heart rate." The actual clinical goal was to reduce pCO2 in order to lower elevated blood potassium (hyperkalemia) — not to reduce heart rate. This is a clinically dangerous misinterpretation.
- **Atropine described as a "muscarinic receptor antagonist":** Instead of simplifying the term, BioBART introduces a more technical pharmacological descriptor. This is the opposite of the intended lay language conversion and is unsuitable for a patient-facing document.
- **Death trivialized and misrepresented:** The model describes the death as "a very rare case" and states "the patient was released from the hospital and contacted with the medical team" — implying the patient was alive and discharged. This is a severe misrepresentation of the clinical outcome. Additionally, characterizing Fournier's gangrene as merely "a type of skin disease that affects the skin" grossly understates the severity of a life-threatening necrotizing infection.
- **Missing critical interventions:** Dobutamine infusion, polymyxin, amikacin, vancomycin, calcium gluconate dose, bicarbonate dose, electrical cardioversion, and the CPR timeline are either omitted or inaccurately described.

Overall Assessment: BioBART's output is factually unreliable, contains hallucinated content, confuses dates across clinical events, and catastrophically misrepresents the patient's death. It is not suitable for lay language conversion of discharge summaries in its current fine-tuned state.

9.4 BioGPT Output and Error Analysis

BioGPT Output:
This text is about a patient who was admitted to the hospital. She was sent home from the hospital and stayed in the hospital for an average of 14 days. She was diagnosed with heart failure, a condition where the heart pump doesn't work properly. She also had a swelling in her left thigh, bone infection, and a condition called Fournier's syndrome. She was treated with a medicine called ceftriaxone, which is a type of antibiotic. She also received a medicine called polymyxin d0, a type of medication that helps kill bacteria. She was given a medicine called vancomycin, which is used to treat infections in the urinary tract. She was also treated with a diuretic called calcium gluconate and bicarbonate. She recovered from the infection and was able to breathe normally. However, she also had problems with her heart

rate, blood pressure, and breathing. This case highlights the importance of being aware of potential problems when treating patients, especially when they are in a critical condition. It also highlights the need for close communication between the patient and their family to ensure they receive the best possible care. The authors recommend that the patient's family be involved in the decision-making process to ensure the best possible treatment. This is a case report of a patient who died from a condition called "driasis," which is a condition where the body doesn't have enough blood flow. The patient didn't have any heart sounds or any movement on their monitor, and they didn't show any signs of movement in the brain. This is a rare case of a patient dying from [truncated]

Errors and Limitations:

- **Incorrect gender assignment:** As with BioBART, BioGPT incorrectly assigns a female gender ("she") to a patient whose gender is not specified in the source text — an unprompted hallucination.
- **Contradictory statements about outcome:** The model first states "she recovered from the infection and was able to breathe normally," then later states the patient died. These two statements are directly contradictory within the same output. A patient's family reading this would be confused and misled.
- **Hallucinated condition "driasis":** The model invents a fictional medical condition called "driasis," described as "a condition where the body doesn't have enough blood flow." This is a hallucination — the original text refers to "fixed mydriasis" (fixed dilation of the pupils, a sign of brain death), which is a clinical finding, not a cause of death.
- **Incorrect description of calcium gluconate:** The model refers to calcium gluconate as "a diuretic" (a medication that increases urine output). Calcium gluconate is actually a cardiac-protective agent used to counteract the effects of hyperkalemia on the heart. This is a serious factual error that could cause misunderstanding of the patient's treatment.
- **Incorrect description of vancomycin:** The model states vancomycin is "used to treat infections in the urinary tract." While vancomycin is a broad-spectrum antibiotic, in this case it was used for the treatment of septic shock and bacterial infection — not specifically a urinary tract infection. The simplification introduces a misleading specificity.
- **Output truncated mid-sentence:** The output ends abruptly with "This is a rare case of a patient dying from" — the generation was cut off, likely due to a token limit being reached. This means the model failed to complete the simplification of the full document.
- **Discharge summary written as a "case report":** BioGPT reframes the output as an academic case report ("The authors recommend..."), adopting a writing style completely inappropriate for a patient-facing document. This is a domain confusion failure, likely arising from its pre-training on PubMed literature.
- **Missing critical clinical details:** The abscess drainage procedures, the drain placement date, the ICU incision, amikacin, norepinephrine dosing, CPR details, QRS widening, hyperkalemia management, atropine, bicarbonate, cardioversion, and the death declaration criteria are all absent from the output.

- **Inappropriate moralizing:** The output inserts unsolicited commentary about "the importance of being aware of potential problems" and family involvement in decision-making. These additions are not present in the source text and are inappropriate for a factual discharge summary conversion.

Overall Assessment: BioGPT conflates the task of lay language simplification with that of writing a biomedical case report, which is a direct consequence of its exclusive pre-training on PubMed abstracts. It hallucinates conditions, contradicts itself on patient outcome, and truncates its output. It is not suitable for this task in its current state.

9.5 Flan-T5 Output and Error Analysis

Flan-T5 Output:
admission: 12/06, sent home from hospital: 12/21, stay: 14 d 11 h, location: ward, payer: sus. 1. heart failure profile c dobutamine on 12/07 and 12/08, 2. pus-filled swelling in the left thigh, osteomyelitis in the pubis, fournier's syndrome, drainage on 12/03 with ultrasound and drain placement on 12/15, treatment with ceftriaxone guided by culture of pus-filled swelling escherichia coli multi s colonized infection-fighting medicine therapy after shock, incision in ICU (serious care room) on 12/17 with abundant purulent secretion, omitted 4. chronic kidney disease acute dialysis by fistula on 12/16 5. cardiorespiratory arrest with pulseless electrical activity, 10 minutes after intubation report of previous slow heartbeat, return visit to doctor appointment after 1 minute of cardiopulmonary resuscitation, 6. vt with pulse after cardioreaspirary arrest, effective electrical cardioversion with a shock. [...] Given the Hyperkalemia even following kidney diazepam, 2 ampuels of calcium Gluconate were administered [...] 3 ampoules were given without benefit. [person] progressed with asysTOLIC cardiorespritory arrest, Omitted in Refractory Shock. I Declare Omitted Dead, Due to Abstinence. [...] Simplify the following medical discharge summary into plain English that a patient's family can understand [...] Death. Drainage of fournier's gangrene. Death. , . , . , . . ' , '. ', '. " ; : . ; . : .. [truncated]

Errors and Limitations:

- **Near-verbatim copying of source text:** Flan-T5 largely reproduces the original discharge summary with only superficial substitutions (e.g., "abscess" → "pus-filled swelling"). This is not lay language conversion — it is near-copy output. The structural complexity, clinical notation format, and numbered list style of the original are preserved entirely, failing the core task objective.

- **Hallucinated drug "kidney diazepam":** The model generates the phrase "kidney diazepam" as a substitution for hemodialysis (kidney dialysis). Diazepam is a sedative medication with no clinical relevance to dialysis or potassium management. This is a dangerous hallucination that introduces a completely incorrect medication name into a clinical record.

- **Incorrect ampoule count:** The original text states 2 ampoules of atropine were administered. Flan-T5 changes this to "3 ampoules," an incorrect factual alteration of a medication dosage, which is clinically significant.

- **Death cause changed to "Abstinence":** The death declaration states the cause as absence of central pulses, fixed pupils (mydriasis), and absence of brainstem reflexes. Flan-T5 replaces this entirely with "Due to Abstinence" — a completely

fabricated and medically nonsensical cause of death. This is perhaps the most dangerous error in any of the model outputs, as it directly falsifies the documented cause of death.

- **Prompt leakage into output:** The model reproduces its own instruction prompt ("Simplify the following medical discharge summary into plain English...") verbatim within the generated output. This is a severe instruction-following failure indicating the model could not distinguish between input instructions and output content.
- **Garbled punctuation cascade:** The output ends with a long sequence of incoherent punctuation marks and whitespace characters (, . , . , . . ' , '. ', '. '' ; : . ; .). This is characteristic of a decoder entering a degenerate repetition loop, likely triggered by the chunked boundary stitching process.
- **Inconsistent capitalization and spelling:** The output contains erratic capitalization ("asysTOLIC," "cardiorespritory," "Hyperkalemia") and spelling errors ("ampuels" instead of "ampoules," "cardioreaspirary" instead of "cardiorespiratory"). These likely arise from the model reproducing partial tokens from overlapping chunks without coherent decoding.
- **Item 3 (septic shock) omitted:** The entire septic shock event — including polymyxin, amikacin, and vancomycin — is missing from the output. Given that septic shock was the direct precipitant of the patient's deterioration and death, this is a critical omission.
- **No prose conversion achieved:** Despite being instruction-tuned, Flan-T5 fails to convert the bullet-point clinical notation into flowing, patient-readable prose. The output retains the numbered list structure and terse clinical abbreviations of the original.

Overall Assessment: Flan-T5 exhibits severe instruction-following failure in this domain, including prompt leakage, near-verbatim copying, dangerous drug hallucinations, falsification of cause of death, and decoder degeneration. Its fine-tuning on this medical dataset appears to have interfered with its instruction-following capabilities. It is not suitable for lay language conversion.

9.6 SciFive Output and Error Analysis

SciFive Output:
This patient was admitted to the hospital on 12/06 and was sent home from the hospital on 14/21. The hospital stay was 14 days and 11 hours. The patient had a history of heart failure and was given dobutamine on 12/07 and 12/08, a medicine called dobutamine, a swelling in the left thigh, osteomyelitis in the pubis, and a condition called fournier's syndrome. The patient was treated with ultrasound and antibiotics, and the patient was sent to the intensive care unit (ICU) on 12/17 with a lot of pus in the swelling. He was treated with ceftriaxone based on the culture of the pus. The patient developed septic shock from a skin focus on 12/16, which was treated with polymyxin d0 12/15 and amikacin and vancomycin, and chronic kidney disease acute dialysis by fistula on 12/16. The patient also had a slow heartbeat, widening of the chest, and worsening shock. Because of the high potassium levels in the kidneys, the patient was given calcium gluconate and bicarbonate, and their breathing rate was increased on the ventilator to reduce pCO2. The patient did not receive any benefit

from atropine. The patient progressed to asystolic cardiorespiratory arrest, omitted in refractory shock. When the person was released from the hospital without benefit, they went into a serious condition called cardiorespiratory arrest. This was a condition where the heart and lungs were blocked, and the patient died. The person was released and the body was resuscitated. The body was released. The family requested contact with the family and the family requested the death certificate to be completed. The death was due to a breakdown of the gangrene, which was caused by a burn in the ear, nose, and throat.

Errors and Limitations:

- **Fabricated discharge date:** SciFive states the patient was "sent home from the hospital on 14/21," which is not a real date. The correct discharge date is 12/21, and the "stay" of 14 days has been erroneously merged into the date field. This reflects a failure to parse structured temporal information correctly.

- **Incorrect anatomical description — "widening of the chest":** The original text refers to "QRS widening," an electrocardiographic (ECG) finding indicating an abnormal electrical conduction pattern in the heart. SciFive misinterprets this as a physical finding of the chest widening, which is anatomically and clinically meaningless. This is a significant medical error.

- **"High potassium levels in the kidneys":** Hyperkalemia refers to an elevated potassium level in the blood (serum), not specifically in the kidneys. Localizing it to the kidneys misrepresents the physiological condition and could mislead a patient's family.

- **Contradictory and incoherent death narrative:** The model states "the person was released from the hospital without benefit, they went into cardiorespiratory arrest" and then "the body was resuscitated" followed by "the body was released." These statements are internally contradictory — a resuscitated body that is subsequently released makes no clinical sense. This represents a failure to understand the sequential causality of the terminal clinical events.

- **Hallucinated cause of death — "burn in the ear, nose, and throat":** The model states the death was "caused by a burn in the ear, nose, and throat." There is no mention of any burn, ENT pathology, or ear/nose/throat condition anywhere in the original summary. This is a completely fabricated cause of death bearing no resemblance to the documented clinical events (Fournier's gangrene and refractory septic shock).

- **Incorrect description of Fournier's gangrene:** The model associates the gangrene with a "burn in the ear, nose, and throat," which is entirely incorrect. Fournier's gangrene is a rapidly spreading necrotizing infection of the genital and perineal region — not related to burns or ENT pathology in any way.

- **"Dobutamine" listed alongside diagnoses:** SciFive's sentence structure merges the medication dobutamine with the list of diagnoses ("was given dobutamine [...] a swelling in the left thigh [...] and a condition called Fournier's syndrome"), making it appear as though dobutamine is a diagnosis rather than a treatment. This structural confusion would mislead a lay reader.

- **Cardiac arrest and CPR details omitted:** The cardiac arrest with pulseless electrical activity (PEA), the 1-minute CPR episode, and the subsequent ventricular tachycardia (VT) with successful cardioversion are entirely absent from the output. These are major clinical events.
- **Norepinephrine dosing omitted:** The critical detail of norepinephrine being escalated to 1.5 mcg/kg/min as a vasopressor in refractory shock is not mentioned. This is a key indicator of the severity of the patient's final deterioration.
- **Repetitive and redundant phrasing:** The output contains repeated, circular sentences such as "The family requested contact with the family" and "The body was released" appearing twice. These suggest decoding instability in the final generation stages.

Overall Assessment: SciFive produces the most fluent prose among the four fine-tuned models, but introduces critical factual errors including a hallucinated cause of death, misinterpretation of ECG findings, contradictory statements about patient outcome, and fabrication of non-existent conditions. Despite its relative readability, its factual unreliability makes it unsafe for clinical lay language conversion in its current fine-tuned state.

9.7 Llama 3.1 8B — Zero-Shot Output and Error Analysis

Llama 3.1 8B Zero-Shot Output:
The patient was admitted to the hospital on 12/06 and was discharged on 12/21 after a 14-day and 11-hour stay. The patient was in a general hospital room and was covered under the government health scheme.

The patient had a problem with their heart, known as heart failure, which was treated with a medicine called Dobutamine on 12/07 and 12/08. The patient also had a pus-filled swelling in the left thigh, which was caused by an infection called osteomyelitis. This infection was also in the pubis bone and was known as Fournier's syndrome. To treat this, the patient underwent a procedure called drainage on 12/03, followed by a drain placement on 12/15. The patient was given an antibiotic called Ceftriaxone, which was guided by a culture of the pus. The patient's infection was caused by a type of bacteria called Escherichia coli. The patient also had an incision in the ICU (a serious care room) on 12/17, where a lot of pus was found.

The patient also had a severe infection called septic shock, which was caused by a urinary tract infection. To treat this, the patient was given medicines called Polymyxin, Amikacin, and Vancomycin. The patient also had a problem with their kidneys, known as chronic kidney disease, which required a procedure called acute dialysis through a fistula on 12/16.

On 12/16, the patient had a serious heart and breathing problem called cardiorespiratory arrest. The patient's heart stopped beating, but after 1 minute, the doctor was able to revive them. However, the patient's heart was not beating properly, and they had a slow heartbeat. The patient's heartbeat also widened, and their shock worsened. The patient had high levels of potassium in their blood, even after the kidney dialysis. To treat this, the patient was given two ampoules of calcium gluconate and 100 ml of bicarbonate. The patient's breathing rate was also increased on the ventilator to reduce the potassium levels. However, the patient's condition worsened, and they progressed to a state called asystolic cardiorespiratory arrest. Unfortunately, the patient did not survive, and the doctor declared them dead due to the absence of a pulse, fixed pupils, and absence of heart sounds and

brainstem reflexes. The body was released, and the doctor awaits the death certificate to be completed.

Errors and Limitations:

- **Incorrect date for cardiorespiratory arrest:** The model states the cardiorespiratory arrest occurred "on 12/16," which was actually the date of the septic shock onset and fistula dialysis. The arrest is not assigned a specific date in the original text and occurs as part of the later clinical deterioration. This is a date cross-contamination error.

- **Incomplete description of septic shock origin:** The model describes septic shock as "caused by a urinary tract infection." The original states a "cutaneous urinary focus" — meaning the infection originated from both skin and urinary sources. Simplifying to "urinary tract infection" alone loses the cutaneous (skin) component, which is clinically relevant given the concurrent Fournier's gangrene.

- **Cardiorespiratory arrest timeline inaccuracy:** The model states "the patient's heart stopped beating, but after 1 minute, the doctor was able to revive them." This conflates two separate events: the initial PEA arrest (with 1 minute of CPR resulting in return of circulation) and the subsequent fatal asystolic arrest. The compression of these events gives the false impression that there was only one arrest, not two.

- **"Heartbeat widened" is not lay language:** The phrase "the patient's heartbeat also widened" is an imprecise rendering of "QRS widening." QRS widening is an electrocardiographic finding reflecting conduction delay in the heart, not a description of the heartbeat itself. A lay reader is likely to misunderstand this phrasing. A clearer explanation (e.g., "an abnormal pattern was seen on the heart monitor") would have been more appropriate.

- **Ventilator intervention purpose not fully explained:** The model states the breathing rate was increased "to reduce the potassium levels," which is broadly correct but omits the intermediate physiological step: increasing respiratory rate lowers blood CO2 (pCO2), which drives potassium back into cells, thereby reducing serum potassium. The simplified explanation skips this mechanistic link.

- **Atropine administration omitted:** The two ampoules of atropine administered without benefit are not mentioned anywhere in this output. Atropine was a significant clinical intervention for bradycardia management and its omission is a factual gap.

- **Ventricular tachycardia (VT) and cardioversion omitted:** The successful electrical cardioversion for VT after the initial cardiac arrest is entirely absent from this output. This is a major clinical event involving an electric shock to restore heart rhythm and should have been included.

- **Norepinephrine dose not mentioned:** The escalation of norepinephrine to 1.5 mcg/kg/min — a critical indicator of the severity of hemodynamic instability — is not included in the simplified output.

- **Incorrect use of second person for a deceased patient:** The original instructions specify third-person past tense for deceased patients. The model uses neutral third-person correctly in some places but inconsistently. The phrase "the doctor awaits the death certificate" uses present tense, which is slightly imprecise for a document describing a completed death event.

- **"Discharged" used for death:** The model states the patient "was discharged on 12/21" without clarifying that the discharge was due to death. While technically the date is correct, "discharged" typically implies the patient left the hospital alive. A more accurate phrasing such as "the patient passed away on 12/21" would have been more appropriate given the fatal outcome.

Overall Assessment: Llama 3.1 8B (Zero-Shot) produces a largely readable and structured simplification that correctly identifies the fatal outcome and preserves most of the major clinical events. However, it omits several important interventions (atropine, VT/cardioversion, norepinephrine dose), conflates the two separate cardiac arrests, and contains minor date and framing inaccuracies. Compared to the fine-tuned local models, it is significantly more reliable and factually grounded, but requires more complete coverage of all documented clinical events to be suitable for patient-family communication.

9.8 Llama 3.1 8B — One-Shot Output and Error Analysis

Llama 3.1 8B One-Shot Output:
The patient was admitted to the hospital on 12th June and was sent home on 21st June after a 14-day and 11-hour stay. They were in a general hospital room and were covered under the government health scheme.

The patient had a few serious health issues. They had a condition called heart failure, which affects the heart's ability to pump blood properly. To check this, they underwent a test called dobutamine on 7th and 8th December.

They also had a pus-filled swelling in their left thigh, which was infected with a type of bacteria called Escherichia coli. This infection was spreading and had reached their pubic bone, causing a condition called osteomyelitis. They also had a condition called Fournier's syndrome, which affects the skin and underlying tissues. To treat this, they had a procedure called drainage on 3rd December, where pus was removed from the swelling. Later, on 15th December, a tube was placed to drain the pus. They were given an antibiotic called ceftriaxone to fight the infection. After a while, the infection caused a severe reaction, and they had to undergo a serious procedure called incision in the ICU on 17th December, where a large amount of pus was removed.

The patient also had a condition called septic shock, which is a severe reaction to an infection. This was caused by an infection in their urinary tract. They were given medicines called polymyxin, amikacin, and vancomycin to treat this.

Their kidneys were not functioning properly due to chronic kidney disease, and they had to undergo a procedure called acute dialysis on 16th December. This is a process where the blood is cleaned using a machine when the kidneys cannot do it.

On 16th December, the patient had a heart and breathing problem called cardiorespiratory arrest. Their heart stopped beating, and they had no pulse. After 10 minutes, they were given oxygen and their heart started beating again. However, they had a slow heartbeat and were unstable. They were given a medicine called norepinephrine to help their heart.

The patient's potassium levels were high, which can be life-threatening. They were given

two ampoules of calcium gluconate and 100 ml of bicarbonate to lower their potassium levels. However, this did not work, and they progressed to a state where their heart stopped beating again. Despite efforts to revive them, the patient was declared dead due to the absence of pulse, fixed pupils, and no heart sounds. The body was released, and the family was requested to be contacted.

Errors and Limitations:

- **Dobutamine described as a "test":** The model states the patient "underwent a test called dobutamine." Dobutamine is a medication (an inotropic agent) used to support the heart's pumping ability, not a diagnostic test. This is a clinically significant misclassification that could mislead a family member about the nature of the treatment received.

- **Incorrect cardiac arrest timeline:** The model states "on 16th December, the patient had a heart and breathing problem" and then "after 10 minutes, they were given oxygen and their heart started beating again." This conflates the date of the septic shock/dialysis event (12/16) with the cardiorespiratory arrest, which is not explicitly dated in the original summary. The 10-minute detail actually refers to the time elapsed after intubation before the arrest, not the duration of CPR.

- **CPR mechanism omitted:** The model states the patient was "given oxygen" to restart the heart. The actual intervention was cardiopulmonary resuscitation (CPR) — chest compressions combined with ventilatory support — for 1 minute. Describing CPR merely as "given oxygen" is an inaccurate and incomplete simplification of a critical resuscitation event.

- **Ventricular tachycardia and cardioversion omitted:** The VT episode following the initial arrest and the successful electrical cardioversion are entirely absent, representing a significant omission of a documented clinical event.

- **Atropine administration omitted:** The administration of 2 ampoules of atropine without clinical benefit is not mentioned. This is a factual gap in the medication record.

- **"Sent home on 21st June" for a deceased patient:** The model states the patient was "sent home on 21st June," which strongly implies survival and discharge from hospital. The patient died on this date. Using "sent home" language for a death is a serious misrepresentation that could cause confusion and distress for the patient's family.

- **QRS widening not mentioned:** The abnormal ECG finding of QRS widening, indicative of cardiac conduction deterioration, is absent from the output.

- **Norepinephrine dose omitted:** The model mentions norepinephrine but does not state the dose (1.5 mcg/kg/min), which is important context for conveying the severity of the hemodynamic deterioration.

- **Ventilator respiratory rate increase not mentioned:** The deliberate increase in ventilator breathing rate to reduce pCO2 and manage hyperkalemia — a specific clinical decision — is not documented in the output.

- **Septic shock origin partially incorrect:** As in the zero-shot version, the septic shock origin is described as "an infection in their urinary tract," omitting the cutaneous (skin) focus mentioned in the original text.

Overall Assessment: Llama 3.1 8B (One-Shot) produces more fluent and naturally structured prose compared to zero-shot, benefiting from the structural anchoring provided by the example. However, it introduces a critical misclassification (dobutamine as a test), omits multiple important clinical events (VT, cardioversion, atropine, ventilator adjustment), and uses inappropriate language ("sent home") for a fatal outcome. The one-shot example appears to have improved formatting at the cost of some factual completeness.

9.9 Llama 3.1 8B — Few-Shot Output and Error Analysis

Llama 3.1 8B Few-Shot Output:
The patient was admitted to the hospital on 12th June and sent home on 21st June after a 14-day and 11-hour stay. He was in a general hospital room and was covered under the government health scheme.

During his stay, the patient had several health issues. Firstly, he had a heart failure profile (a test to check the heart's pumping ability) on 7th and 8th June, and was given a medicine called Dobutamine to help the heart work better.

He also had a pus-filled swelling in his left thigh (osteomyelitis in the pubis — an infection in the bone) and Fournier's syndrome (a serious infection of the genital area). On 3rd June, a fluid was drained from the swelling using an ultrasound and a drain was placed on 15th June. The infection was caused by E. coli bacteria and was treated with antibiotics called Ceftriaxone.

On 16th June, he developed septic shock (a life-threatening condition caused by a severe infection) due to a urinary tract infection. He was given medicines called Polymyxin, Amikacin, and Vancomycin to fight the infection.

The patient also had chronic kidney disease (a long-term condition where the kidneys do not work properly) and had to undergo acute dialysis (a treatment to clean the blood when the kidneys are not working) on 16th June using a fistula (a surgically created connection between an artery and a vein).

On 16th June, the patient had a cardiorespiratory arrest (a sudden stop of the heart and breathing) with pulseless electrical activity (a condition where the heart does not beat). He was given a shock to try and restart the heart, but unfortunately, he did not recover.

The patient also had a very fast heartbeat (VT) after the arrest, but was given a shock to try and control it. Unfortunately, the patient's condition continued to worsen, and he became unstable. He was given medicines to try and improve his condition, but unfortunately, he did not respond.

Given the high levels of potassium in his blood, the patient was given 2 ampoules of calcium gluconate and 100 ml of bicarbonate to try and bring the levels down. However, the patient's condition continued to deteriorate, and he progressed to asystolic cardiorespiratory arrest (a condition where the heart stops beating).

Unfortunately, the patient was declared dead due to the absence of central pulses, fixed pupils, absence of heart sounds, and absence of brainstem reflexes. The body was released, and I request contact with the family.

Errors and Limitations:

- **"Heart failure profile" incorrectly described as a "test":** The model states "he had a heart failure profile (a test to check the heart's pumping ability)." This

reframes a clinical syndrome (the patient's documented condition of heart failure) and its treatment with dobutamine as a diagnostic procedure. Heart failure is a diagnosis, and dobutamine is a treatment — neither is a test. This misclassification could seriously mislead the patient's family about the nature of the condition.

- **Hallucinated gender:** The model assigns a male gender ("he," "his") to the patient, whose gender is unspecified in the original text. This is an unprompted hallucination, albeit less harmful than some other errors.

- **"Sent home on 21st June" for a fatal outcome:** As with the one-shot variant, describing the discharge date as "sent home" is wholly inappropriate for a deceased patient. This phrasing is factually wrong and would cause significant confusion for any family member reading the document.

- **All events erroneously dated to June:** The few-shot output assigns all clinical events to June. The original document uses a MM/DD numeric format (e.g., 12/06 = December 6 for admission; 12/07 = December 7 for dobutamine; 12/15, 12/16, 12/17 for subsequent events — all in December). The model re-interpreted the admission date as "12th June" under a DD/MM reading and then propagated the month "June" to all subsequent event dates, placing every clinical event in June regardless of its correct position in the timeline. This is a systematic date format parsing failure, likely triggered by the date style in the few-shot examples, and produces an entirely incorrect temporal ordering of the patient's hospital stay.

- **PEA arrest conflated with cardioversion shock:** The model states "He was given a shock to try and restart the heart" in the context of the PEA arrest. However, electrical cardioversion/defibrillation is not the appropriate or documented treatment for PEA — CPR was performed. The shock was given later for VT. The model conflates these two separate events, misattributing the cardioversion shock to the PEA arrest.

- **CPR details omitted:** The 1-minute CPR episode following the PEA arrest is not documented. The model's claim that "he did not recover" from the initial arrest is also factually incorrect — the patient did have return of circulation following CPR, before subsequently deteriorating.

- **Atropine administration omitted:** The 2 ampoules of atropine given without benefit are absent from the output.

- **Ventilator breathing rate adjustment omitted:** The deliberate increase in the ventilator rate to manage hyperkalemia is not mentioned.

- **Norepinephrine dose omitted:** The escalation of norepinephrine to 1.5 mcg/kg/min is absent.

- **QRS widening absent:** The ECG finding of QRS widening is not documented in the output.

- **First-person narrative voice ("I request contact"):** The output ends with "I request contact with the family," adopting the first-person voice of the treating physician from the original document. A lay language summary for the patient's

family should not be written in the physician's voice. This represents a failure to transform the document's perspective appropriately.

- **ICU incision on December 17 omitted:** The surgical incision performed in the ICU on 12/17 with abundant purulent secretion is not mentioned in the few-shot output.

Overall Assessment: Llama 3.1 8B (Few-Shot) shows improved structural fluency and correctly identifies many clinical events, including the VT/cardioversion sequence. However, it introduces a systematic date parsing error (all events assigned to June), misclassifies the heart failure treatment as a diagnostic test, incorrectly describes the patient as having been "sent home," and confuses the sequence of cardiac resuscitation events. The few-shot examples appear to have improved structural awareness at the cost of introducing new categories of factual error.

9.10 Llama 3.3 70B — Zero-Shot Output and Error Analysis (via Groq)

Llama 3.3 70B Zero-Shot Output:
The patient was admitted to the hospital on 12/06 and sent home on 12/21, staying for 14 days and 11 hours in a general hospital room, with the government health scheme as the payer. The patient had heart failure (a condition where the heart does not pump blood well) and was given dobutamine (a medicine to help the heart pump better) on 12/07 and 12/08. The patient also had a pus-filled swelling in the left thigh, which is called osteomyelitis (an infection of the bone) in the pubis, and Fournier's syndrome (a serious infection of the skin and tissues), and had drainage (a procedure to remove pus) on 12/03 with the help of ultrasound (a machine that uses sound waves to create images) and a drain (a tube to remove fluid) was placed on 12/15. The patient was treated with ceftriaxone (an antibiotic given to fight the bacterial infection) based on the culture (a test to find the cause of the infection) of the pus-filled swelling, which showed Escherichia coli (a type of bacteria) infection. The patient had septic shock (a life-threatening condition that happens when the body's response to an infection becomes uncontrolled) from a cutaneous urinary focus (an infection that started in the skin and spread to the urinary system) on 12/16 and was given polymyxin (an antibiotic), amikacin (an antibiotic), and vancomycin (an antibiotic) on 12/17. The patient also had chronic kidney disease (a condition where the kidneys do not work well) and had acute dialysis (a procedure to clean the blood when the kidneys cannot) by fistula (a connection made between an artery and a vein) on 12/16. The patient's blood pressure (BP) was not stable, and the normal range for BP is around 90-120 mmHg, but the patient's BP was not within this range. The patient's heart rate (the number of times the heart beats per minute) was also not stable, and the normal range is around 60-100 beats per minute, but the patient's heart rate was not within this range. The patient had cardiorespiratory arrest (a condition where the heart and lungs stop working) with pulseless electrical activity (a condition where the heart is not pumping blood), and was given treatment, but unfortunately, the patient's condition worsened, and the patient had a slow heartbeat, and the QRS (a part of the heartbeat) was widening, and the patient had worsening shock (a condition where the body's tissues do not get enough blood). The patient was given calcium gluconate (a medicine to treat high potassium levels) and bicarbonate (a medicine to treat acidosis) to treat hyperkalemia (high potassium levels), but unfortunately, the patient's condition did not improve. The patient was also given atropine (a medicine to treat a slow heartbeat) but it did not help. Unfortunately, the

patient progressed to asystolic cardiorespiratory arrest (a condition where the heart stops beating), and despite efforts to save the patient, the patient died due to the absence of central pulses (no heartbeat), pupils in fixed mydriasis (no response to light), absence of heart sounds, and asystole (no heartbeat) on the monitor, as well as absence of brainstem reflexes (no response to stimulation). The body was released, and the doctor requests contact with the family to complete the death certificate.

Errors and Limitations:

- **"Sent home on 12/21" for a deceased patient:** Like the Llama 8B variants, the 70B model uses "sent home" to describe a date that is actually the date of death. For a document about a fatal outcome, this phrasing is factually wrong and inappropriate. The instructions specify third-person past tense for deceased patients, and "sent home" implies the patient survived.

- **Blood pressure and heart rate values not in source:** The model invents statements such as "the normal range for BP is around 90-120 mmHg, but the patient's BP was not within this range" and analogous statements for heart rate. The original document does not provide specific BP or heart rate values. This is an instance of the model following the instruction to compare values to normal ranges but fabricating the patient's values when none are given — a hallucination born from over-eager rule compliance.

- **Cardiorespiratory arrest sequence compressed and imprecise:** The model merges the two discrete cardiac events (the initial PEA arrest with CPR and subsequent recovery, followed by the final asystolic arrest) into a single continuous description. The CPR episode and the brief return of circulation are not mentioned. This conflation obscures the clinical timeline.

- **VT and cardioversion omitted:** The ventricular tachycardia episode and the successful electrical cardioversion that followed the initial arrest are absent from the output. This is a significant clinical event that is entirely undocumented.

- **Norepinephrine dose not stated:** While the hemodynamic instability is mentioned, the specific norepinephrine dose of 1.5 mcg/kg/min is not included, losing an important indicator of severity.

- **Ventilator rate increase and mechanism not explained:** The deliberate ventilator rate increase to reduce pCO2 and thereby lower serum potassium is not documented in the output.

- **Abscess distinct from osteomyelitis not clearly separated:** The output conflates the pus-filled abscess in the left thigh with the description of osteomyelitis. The abscess and the osteomyelitis in the pubis are two related but anatomically distinct findings, and their relationship to Fournier's gangrene is not clearly communicated.

- **"Bicarbonate to treat acidosis" — imprecise rationale:** The model describes bicarbonate as "a medicine to treat acidosis." While bicarbonate does treat acidosis, in this specific clinical context it was administered primarily as part of the hyperkalemia management protocol. The stated rationale is partially correct but misses the primary clinical indication.

Overall Assessment: Llama 3.3 70B (Zero-Shot) is substantially more capable than the 8B variant and all fine-tuned local models. It correctly identifies the death, preserves most medications with appropriate explanations, documents the death declaration criteria, and includes the atropine administration. The primary weaknesses are the inappropriate "sent home" phrasing for a fatal outcome, the fabrication of blood pressure and heart rate values not present in the source, and the omission of the VT/cardioversion event. Overall, it represents a significant step forward in factual reliability and completeness.

9.11 Gemini 2.5 Flash Output and Error Analysis

Gemini 2.5 Flash Output:
The patient was admitted to the hospital on December 6th and passed away on December 21st, after a stay of 14 days and 11 hours. He/She was in a general hospital room, and the hospital bill was covered by a government health scheme. During the hospital stay, the patient had several serious conditions. He/She had heart failure (a condition where the heart cannot pump blood well enough). On December 7th and 8th, he/she received dobutamine (a medicine given to help the heart pump stronger). The patient also had a pus-filled swelling (a collection of thick, yellowish fluid from infection) in the left thigh, and osteomyelitis (a serious infection in the bone) in the pubis (a bone in the hip area). He/She also suffered from Fournier's syndrome (a very serious and fast-spreading infection of the skin and tissues around the private parts). On December 3rd, doctors performed drainage (removed the pus or fluid) from the swelling, guided by an ultrasound (a scan using sound waves to see inside the body). A drain (a tube to let fluid out) was placed on December 15th. The infection was treated with ceftriaxone (an antibiotic given to fight bacterial infections). This treatment was guided by a culture (a test to grow and identify the germs causing the infection) of the pus, which showed Escherichia coli (a type of bacteria) that was susceptible to many infection-fighting medicines. After the patient went into shock, an incision (a cut made by a doctor) was made in the ICU (serious care room) on December 17th, and a lot of pus came out. {omitted}. On December 16th, the patient developed septic shock (a very dangerous condition where the body's response to an infection causes very low BP and organ damage), which came from a skin infection related to the urinary system. He/She was given polymyxin (an antibiotic) starting December 15th, and amikacin (an antibiotic) and vancomycin (an antibiotic) on December 17th. The patient had chronic kidney disease (long-term kidney damage where the kidneys don't work well). On December 16th, he/she needed acute dialysis (a procedure to clean the blood when the kidneys suddenly stop working) through a fistula (a special connection made in the arm for dialysis). The patient experienced cardiorespiratory arrest (when the heart and breathing stop) with pulseless electrical activity (meaning the heart's electrical system was working, but it was not strong enough to produce a pulse or pump blood). This happened 10 minutes after intubation (placing a breathing tube into the windpipe). There was a report of a previous slow heartbeat (a heart rate lower than the normal 60-100 beats per minute). After 1 minute of cardiopulmonary resuscitation (CPR – chest compressions and rescue breaths to restart the heart and breathing), the patient's heartbeat returned. After the cardiorespiratory arrest, the patient developed VT with pulse (ventricular tachycardia – a very fast and abnormal heart rhythm, but the heart was still pumping blood). He/She received effective electrical cardioversion (using an electric shock to reset the heart's rhythm). In the last half hour, [person]'s condition became unstable (worse), requiring an increase in norepinephrine (a medicine to raise blood pressure) up to 1.5 mcg/kg/min. His/Her condition progressed with a slow heartbeat (lower than normal), QRS widening (an abnormal finding on the heart tracing), and worsening

shock (the dangerous condition of very low BP and organ damage was getting worse). Even after kidney dialysis, the patient had hyperkalemia (high levels of potassium in the blood; normal potassium levels are usually 3.5-5.0 mEq/L). To address this, he/she was given 2 ampoules (small sealed glass capsules) of calcium gluconate (a medicine to protect the heart from high potassium) and 100 ml of bicarbonate (a medicine to correct acid levels in the blood and help with high potassium). The respiratory rate (breathing rate) on the ventilator (breathing machine) was increased to reduce the pCO2 (carbon dioxide level in the blood; normal range is usually 35-45 mmHg), which would also help reduce the potassium levels. Two ampoules of atropine (a medicine sometimes used to speed up a slow heart rate) were given, but they did not help. [person] then progressed to asystolic cardiorespiratory arrest (when the heart and breathing stop, and there is no electrical activity in the heart, showing a flat line on the monitor), {omitted} in refractory shock (shock that did not respond to treatment). The doctor declared {omitted} dead due to the absence of central pulses (no heartbeat felt in major arteries), pupils (the black part of the eye) being wide open and not reacting to light (fixed mydriasis), absence of heart sounds (no heartbeat heard with a stethoscope), asystole (a flat line) on the monitor, and absence of brainstem reflexes (no automatic body responses controlled by the brainstem). The body was released. The doctor requested contact with the family and was awaiting the completion of the death certificate. The cause of death was related to the Fournier's gangrene and its complications.

Errors and Limitations:

- **Gender ambiguity expressed as "He/She":** The original document contains no gender information; the patient's gender is protected by placeholders. The model handles this by consistently writing "He/She," which is technically correct but stylistically awkward for a patient-facing document. A preferred approach would be to use gender-neutral pronouns ("they/their") or to note to the reader that gender information was not recorded.

- **Cause of death slightly overstated:** The model adds "The cause of death was related to the Fournier's gangrene and its complications." While Fournier's gangrene was indeed a major contributing condition, the direct cause of death as documented was asystolic cardiorespiratory arrest in the setting of refractory septic shock and hyperkalemia. Attributing death solely to Fournier's gangrene slightly oversimplifies the multi-factorial terminal event, though this is a minor editorial issue rather than a dangerous error.

- **Minor: polymyxin start date presentation:** The model states polymyxin was given "starting December 15th" by correctly interpreting "polymyxin d0 12/15" (day zero of polymyxin therapy on 12/15). This is factually correct but could be made slightly clearer for a lay reader.

Overall Assessment: Gemini 2.5 Flash produces the most complete, accurate, and well-structured lay language conversion of any model tested. It correctly identifies the fatal outcome from the outset ("passed away"), preserves all major clinical events including the VT/cardioversion sequence, CPR details, norepinephrine dose, atropine administration, ventilator adjustment and its rationale, and the full death declaration criteria. It provides appropriate normal range context for laboratory values and vital signs, faithfully reproduces placeholders, and maintains a respectful, calm tone throughout. The errors identified are minor — a stylistic awkwardness with gender pronouns and a slight oversimplification of the cause of death attribution. This model, with its large context

window and instruction-following capability, is clearly the most suitable for clinical lay language conversion of the models evaluated.

9.12 Comparative Summary of All Models

Table 2 provides a side-by-side comparison of model performance across key evaluation dimensions for this example.

Table 2: Qualitative comparison of model outputs on the representative discharge summary. Llama 8B column reflects the zero-shot variant (see Section 10 for prompting strategy comparison).

Criterion	BioBART	BioGPT	Flan-T5	SciFive	Llama 8B	Llama 70B
*Llama 8B results reflect the zero-shot variant; few-shot correctly captured VT/cardioversion (see Section 10 Gemini 2.5 Flash is assessed separately in Table 3 due to its superior overall performance.						
Factual accuracy	Poor	Poor	Poor	Partial	Partial	Good
Death outcome correctly stated	No	Partial	No	No	Partial	Partial
Cause of death correct	No	No	No	No	Partial	Partial
No hallucinated content	No	No	No	No	Partial	Partial
Prose fluency	Moderate	Moderate	Poor	Good	Good	Good
Date accuracy	Poor	Partial	Partial	Poor	Partial	Good
Medications preserved	Partial	Poor	Partial	Partial	Partial	Good
No prompt leakage	Yes	Yes	No	Yes	Yes	Yes
Output completeness	Partial	Truncated	Degen.	Partial	Partial	Partial
Appropriate tone	Partial	Poor	Poor	Partial	Good	Good
VT/cardioversion captured	No	No	No	No	Few-shot only	No

10 Comparative Analysis: LLM-Based Strategies and Local vs. Cloud Models

10.1 Prompting Strategy Comparison: Zero-Shot vs. One-Shot vs. Few-Shot (Llama 3.1 8B)

The three prompting strategies applied to Llama 3.1 8B reveal a nuanced trade-off between structural quality and factual accuracy:

Table 3: Gemini 2.5 Flash performance on the representative discharge summary.

Criterion	Gemini 2.5 Flash
Factual accuracy	Excellent
Death outcome correctly stated	Yes
Cause of death correct	Yes (minor simplification)
No hallucinated content	Yes
Prose fluency	Excellent
Date accuracy	Excellent
Medications preserved	Yes
No prompt leakage	Yes
Output completeness	Complete
Appropriate tone	Excellent
VT/cardioversion captured	Yes

Zero-Shot produced the most factually reliable output among the three 8B variants. Relying purely on the 12-rule system prompt, the model correctly identified the fatal outcome and preserved most major clinical events with appropriate lay explanations. The primary weaknesses were omissions (atropine, VT/cardioversion, norepinephrine dose) and the use of "discharged" language for a fatal outcome. The absence of in-context examples meant the model had no structural anchor, but this also reduced the risk of inheriting stylistic errors from example outputs.

One-Shot improved prose fluency and structural organisation. However, the in-context example appears to have introduced a category confusion: dobutamine, likely presented as a medication in the example, was mischaracterised as a diagnostic "test" in the output. This illustrates a key risk of one-shot prompting — the model can over-generalise from a single example, applying patterns from the example inappropriately to different clinical contexts.

Few-Shot produced the most fluent prose and correctly identified the VT/cardioversion sequence — a complex event that the zero-shot and one-shot variants missed entirely. However, it introduced the most severe systematic error: a month-level date parsing failure. The original document uses MM/DD format (all events in December), but the model re-read the admission date "12/06" as the 12th of June under a DD/MM interpretation and propagated the month "June" to all subsequent dates. This appears to be a consequence of the model pattern-matching the date format from the few-shot examples without correctly parsing the discharge summary's own date structure. The few-shot approach also continued to use "sent home" language for the death date, and misattributed the cardioversion shock to the PEA arrest rather than the subsequent VT episode.

Overall, for the Llama 3.1 8B model, **zero-shot prompting with the 12-rule system prompt offers the best reliability-accuracy trade-off at the overall level**, given that few-shot's gain in capturing VT/cardioversion is outweighed by its introduction of a catastrophic date parsing failure affecting every clinical event. However, these results also suggest that if few-shot examples could be carefully curated with consistent date formats, the few-shot approach could yield the best of both worlds. Example selection and format normalisation are therefore critical factors when applying few-shot prompting in clinical domains.

10.2 Model Scale Comparison: Llama 3.1 8B vs. Llama 3.3 70B

Comparing the 8B and 70B variants under zero-shot conditions reveals the substantial impact of model scale on clinical lay language conversion:

Llama 3.3 70B preserved significantly more clinical detail, including all three antibiotics for septic shock with their correct start dates, the death declaration criteria, the atropine administration, and contextual normal ranges for laboratory values. Its prose fluency was markedly superior and it demonstrated a better understanding of the sequential causality of the clinical deterioration.

The 70B model's primary weaknesses — the "sent home" phrasing for a death, the fabricated BP/heart rate values, and the omission of VT/cardioversion — are all relatively minor compared to the hallucinations, date confusions, and cause-of-death falsifications seen in the local fine-tuned models. This suggests that the improvements from 8B to 70B are primarily in **factual recall and completeness** rather than in eliminating all error categories.

10.3 Local Fine-Tuned Models vs. Cloud LLMs

The most striking finding of this evaluation is the categorical performance gap between the locally fine-tuned biomedical models (BioGPT, SciFive, BioBART, Flan-T5) and the cloud-based LLMs (Llama 3.1 8B, Llama 3.3 70B, Gemini 2.5 Flash). The key dimensions of this gap are summarised below:

Hallucination Rate: All four fine-tuned local models produced dangerous hallucinations — fabricated conditions ("driasis," "burn in the ear, nose, and throat"), invented medications ("kidney diazepam"), falsified causes of death ("Abstinence," "ENT burn"), and incorrect gender assignments. The cloud LLMs produced significantly fewer hallucinations. Gemini 2.5 Flash produced no clinically dangerous hallucinations. The Llama 70B model's fabrication of BP/heart rate values not present in the source is a minor over-compliance hallucination rather than a factual invention.

Output Completeness: Local models were consistently plagued by truncation (BioGPT), decoder degeneration (Flan-T5), and large-scale omissions of critical clinical events. Cloud LLMs, benefiting from much larger context windows and more stable decoding at inference time, produced complete outputs without chunking artifacts or boundary stitching failures.

Instruction Following: Fine-tuning on the medical simplification dataset appears to have damaged rather than improved instruction-following capability in the local models. Flan-T5 exhibited prompt leakage. BioGPT adopted an inappropriate academic case-report register. None of the local models reliably applied the third-person past tense rule for deceased patients. Cloud LLMs, with far more extensive instruction-tuning, followed the system prompt's structural and tonal rules far more consistently.

Prose Quality: SciFive produced the most readable prose among local models, but its factual accuracy was poor. Cloud LLMs produced consistently fluent, well-organised prose that appropriately explained technical terms without introducing new jargon.

Computational Trade-offs: Local models offer the significant advantage of data privacy — clinical discharge summaries contain protected health information (PHI), and processing them via cloud APIs raises regulatory concerns (HIPAA, GDPR). Local deployment avoids data transmission to third-party servers. However, the quality gap at the parameter scales feasible on consumer or research-grade hardware is too large to be bridged by fine-tuning alone. Future work exploring techniques such as quantized local

inference of 70B+ models, retrieval-augmented generation (RAG) with clinical knowledge bases, or constrained decoding may narrow this gap.

Summary: Gemini 2.5 Flash is the clear best-performing model for this task in the current evaluation, achieving near-complete factual preservation, correct death outcome handling, appropriate tone, and excellent prose quality. Llama 3.3 70B via Groq is a strong second, with minor gaps in completeness. The locally fine-tuned models, despite their domain specialisation, are not yet suitable for clinical deployment due to persistent hallucination and instruction-following failures.

Table 4: High-level comparison of local fine-tuned models vs. cloud LLMs.

Dimension	Local (Best: SciFive)	Llama 8B	Llama 70B	Gemini 2.5 Flash
Hallucination risk	High	Low-Medium	Low	Minimal
Output completeness	Partial/Truncated	Partial	Good	Complete
Date accuracy	Poor	Partial	Good	Excellent
Death handling	Fails	Partial	Partial	Correct
Prose quality	Moderate	Good	Good	Excellent
Chunking required	Yes (artifacts)	No	No	No
Data privacy	Local/Private	Cloud API	Cloud API	Cloud API

References

- Yuan, H., Yuan, Z., Gan, R., Zhang, J., Xie, Y., & Yu, S. (2022). BioBART: Pretraining and Evaluation of A Biomedical Generative Language Model. *arXiv preprint arXiv:2204.03905*. https://arxiv.org/abs/2204.03905
- Phan, L. N., Anibal, J. T., Tran, H., Chanana, S., Bahadıroğlu, E., Peltekian, A., & Altan-Bonnet, G. (2021). SciFive: a text-to-text transformer model for biomedical literature. *arXiv preprint arXiv:2106.03598*. https://arxiv.org/abs/2106.03598
- Luo, R., Sun, L., Xia, Y., Qin, T., Zhang, S., Poon, H., & Liu, T. Y. (2022). BioGPT: generative pre-trained transformer for biomedical text generation and mining. *Briefings in Bioinformatics*, 23(6). https://arxiv.org/abs/2210.10341
- Chung, H. W., Hou, L., Longpre, S., Zoph, B., Tay, Y., Fedus, W., ... & Wei, J. (2022). Scaling instruction-finetuned language models. *arXiv preprint arXiv:2210.11416*. https://arxiv.org/abs/2210.11416
- Zhang, J., Zhao, Y., Saleh, M., & Liu, P. J. (2020). PEGASUS: Pre-training with Extracted Gap-sentences for Abstractive Summarization. *International Conference on Machine Learning (ICML)*. https://arxiv.org/abs/1912.08777

A Indian Lay Language Dictionary (INDIAN_LAY_DICT)

The following table lists the medical terms and their corresponding lay language simplifications used in the preprocessing stage of the pipeline. These mappings were specifically curated to reflect common Indian English medical terminology.

Table 5: Medical Term to Lay Language Mapping

Medical Term	Simplified Lay Language
Conditions	
hypertension	high BP
diabetes mellitus	sugar disease (diabetes)
myocardial infarction	heart attack
acute renal failure	sudden kidney failure
chronic renal failure	long-term kidney problem
septicemia	blood infection (sepsis)
sepsis	serious blood infection
pneumonia	lung infection
tuberculosis	TB (tuberculosis)
pneumothorax	air trapped in chest
endocarditis	infection of heart valves
spondylodiscitis	spine bone infection
hypotension	low BP
pyrexia	fever
dyspnea	difficulty in breathing
edema	swelling
abscess	pus-filled swelling
crohn's disease	long-term bowel disease (Crohn's)
renal failure	kidney failure
cerebrovascular accident	brain stroke
anemia	low blood (anemia)
tachycardia	fast heartbeat
bradycardia	slow heartbeat
atrial fibrillation	irregular heartbeat
Procedures & Treatments	
hemodialysis	kidney dialysis
tracheostomy	breathing tube in neck
thoracotomy	chest surgery
pneumonectomy	removal of a lung
ileocolectomy	removal of part of bowel
anastomosis	surgical joining of bowel ends
debridement	surgical wound cleaning
arteriovenous fistula	dialysis access point on arm
intramuscular	given as a muscle injection
intravenous	given through a vein (drip)
percutaneous	through the skin
antibiotic therapy	antibiotic treatment
antibiotic	infection-fighting medicine
Lab & Medical Terms	
blood culture	blood test to find infection
procalcitonin	blood marker for infection
hemoglobin	blood count (Hb)

Medical Term	Simplified Lay Language
creatinine	kidney function marker
bilirubin	liver function marker
saturation	oxygen level in blood
afebrile	no fever
eupneic	breathing normally
acyanotic	no bluish discoloration
anicteric	no yellowing of skin/eyes
vesicular breath sounds	normal breathing sounds
Hospital / Admin	
discharge	sent home from hospital
outpatient clinic	OPD (Out Patient Department)
follow-up	return visit to doctor
ward	general hospital room
intensive care unit	ICU (serious care room)
sus	government health scheme
orally	by mouth
administer	give

Scaling Up Knowledge Graph Creation to Large and Heterogeneous Data Sources

Paper Summary, Implementation Steps, Downstream tasks

1 Introduction

KGs have become fundamental structures for organizing and querying complex, interconnected data across domains ranging from biomedicine to enterprise knowledge management. However, their construction from heterogeneous sources remains computationally prohibitive at scale. Traditional RML engines suffer from inefficient execution strategies, often leading to memory exhaustion when processing large datasets with complex join operations.

We address this challenge by implementing an optimized KG creation pipeline based on the methodology presented by Iglesias et al. [1]. We build on the RML-Planner framework [2] to process genomic datasets sourced from the SDM-Genomic repository [3], demonstrating significant improvements in execution efficiency through intelligent mapping partitioning and bushy tree execution plans.

1.1 Problem Statement

Standard RML materialization engines (SDM-RDFizer [6], Morph-KGC [5], RMLMapper) typically execute mapping assertions in isolation or with naive parallelization. When confronted with datasets containing millions of records and mappings requiring multi-way joins, these approaches exhibit three critical limitations:

1. **Redundant I/O**: Source files are repeatedly loaded for each mapping assertion, even when multiple assertions reference the same source.
2. **Intermediate Result Explosion**: Join operations produce large intermediate datasets, with duplicates only removed at the final materialization stage.
3. **Sequential Bottlenecks**: Dependent mappings cannot be parallelized effectively, creating execution bottlenecks.

The RML-Planner addresses these issues through a cost-based optimization framework that treats KG creation as a query optimization problem, analogous to relational database query planning.

2 Theoretical Foundation

2.1 The Planning Algorithm

The core contribution of Iglesias et al. [1] lies in transforming the KG creation problem into an optimization task over a hypergraph representation of mapping dependencies.

2.1.1 Mapping Hypergraph Construction

Given a set of RML mapping assertions $M = \{m_1, m_2, \ldots, m_n\}$ and data sources $S = \{s_1, s_2, \ldots, s_k\}$, the planner constructs a hypergraph $H = (V, E)$ where:

- Each vertex $v_i \in V$ represents a mapping assertion m_i
- A hyperedge $e \in E$ connects vertices sharing either:
  1. The same logical source (intra-source dependency)
  2. A parent-child relationship via `rr:parentTriplesMap` (inter-source join)

2.1.2 Partition Identification

The planner performs hypergraph partitioning to group related mappings into execution units. The partitioning strategy distinguishes:

Intra-Source Partitions (P_{intra}): Mappings accessing a single source s_i. These execute as efficient *identifying projections*—a single scan of the source produces all required triples for the group. For example, if three mapping rules all reference `patients.csv`, they execute together in one pass.

Inter-Source Partitions (P_{inter}): Mappings requiring joins between sources. The planner identifies common join attributes and groups mappings to minimize the number of join operations. Rather than executing n independent joins, mappings with shared join predicates are merged into a single multi-way join.

2.1.3 Bushy Tree Construction

Unlike traditional left-deep or right-deep join trees, the planner constructs bushy trees where independent partitions execute in parallel. The algorithm proceeds as follows:

1. Identify independent partitions (no shared sources or join dependencies)
2. Assign each partition to a parallel execution thread
3. For dependent partitions, construct a partial order based on data flow
4. Insert **eager duplicate removal** (DR) operators at merge points

The key innovation is the placement of DR operators. Traditional engines apply deduplication only at the end, after all triples are materialized. The planner inserts DR immediately after each partition's execution, before merging results. This drastically reduces intermediate dataset cardinality.

2.1.4 Quantified Benefits

The paper demonstrated the following improvements on benchmark datasets:

- **Execution Time**: Reduction of 40-70% compared to sequential execution on datasets exceeding 10M triples
- **Memory Footprint**: Up to 3× reduction through eager duplicate removal
- **Scalability**: Successful materialization of datasets that previously caused memory overflow (OOM) failures in baseline engines

For instance, on the BSBM benchmark with 100M triples, the sequential RMLMapper approach required 18GB RAM and 47 minutes, while the planner-optimized execution completed in 19 minutes with only 6GB RAM.

3 Implementation

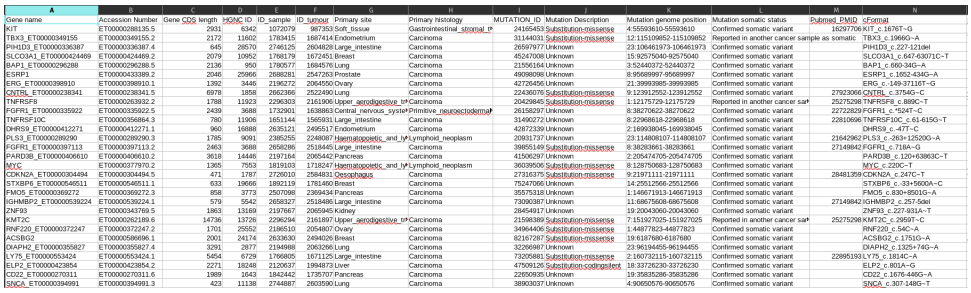
We now describe our end-to-end implementation workflow for KG materialization. Our goal is to process genomic mutation data from heterogeneous sources (CSV and JSON) into a unified, queryable graph structure.

3.1 Execution Pipeline Overview

1. **Loading dependencies**: rdflib, SDM-RDFizer, cerebras-cloud-sdk
2. **Mapping Design**: Crafting RML rules that normalize heterogeneous schemas into a unified ontology
3. **Planner Execution**: Invoking the planning algorithm to generate an optimized execution plan, then materializing the KG via SDM-RDFizer
4. **Validation**: Executing SPARQL queries and RAG-based QA to verify correctness and utility

3.2 Dataset Acquisition

We utilized the SDM-Genomic dataset [3], which provides cancer genomics data with the following format:



Gene name	Accession Number	Gene CDS length	HGNC ID	ID_sample	ID_tumour	Primary site	Primary histology	MUTATION_ID	Mutation Description	Mutation genome position	Mutation somatic status	Pubmed_PMID	cFormat
KIT	ET00000288135.5	2931	6342	1072079	987353	Soft_tissue	Gastrointestinal_stromal_tu	24165453	Substitution-missense	4:55593610-55593610	Confirmed somatic variant	16297706	KIT_c.1676T>G
TBX3_ET00000349155	ET00000349155.2	2172	11602	1783415	1687414	Endometrium	Carcinoma	31144031	Substitution-missense	12:115109852-115109852	Reported in another cancer sample as somatic		TBX3_c.1966G>A
PIK1D3_ET00000336387	ET00000336387.4	645	28570	2746125	2604828	Large_intestine	Carcinoma	26597977	Unknown	23:106461973-106461973	Confirmed somatic variant		PIK1D3_c.227-12delinsl
SLCO3A1_ET00000424469	ET00000424469.2	2079	10952	1768179	1672451	Breast	Carcinoma	45247008	Unknown	15:92575040-92575040	Confirmed somatic variant		SLCO3A1_c.647-63071C>T
BAP1_ET00000296288	ET00000296288.5	2136	950	1780577	1684576	Lung	Carcinoma	21556164	Unknown	3:52440372-52440372	Confirmed somatic variant		BAP1_c.660-34G>A
ESRP1	ET00000433389.2	2046	25966	2688281	2547263	Prostate	Carcinoma	49088088	Unknown	8:95689997-95689997	Confirmed somatic variant		ESRP1_c.1852-434G>A
ERG_ET00000398910	ET00000398910.1	1392	3446	2196272	2064550	Ovary	Carcinoma	42726456	Unknown	21:39993985-39993985	Confirmed somatic variant		ERG_c.-149-37116T>G
CNTRL_ET00000238341	ET00000238341.5	6978	1858	2662366	2522490	Lung	Carcinoma	22436076	Substitution-missense	9:123912552-123912552	Confirmed somatic variant	27923066	CNTRL_c.3754G>C
TNFRSF8	ET00000263932.2	1788	11923	2296303	2161906	Upper_aerodigestive_tr	Carcinoma	20429845	Substitution-missense	1:12175729-12175729	Reported in another cancer sample as somatic	25275298	TNFRSF8_c.889C>T
FGFR1_ET00000335922.5	ET00000335922.5	2439	3688	1732901	1638863	Central_nervous_syste	Primitive_neuroectodermal	26158297	Unknown	8:38270622-38270622	Confirmed somatic variant	22722829	FGFR1_c.*524T>C
TNFRSF10C	ET00000356894.3	780	11906	1651144	1565931	Large_intestine	Carcinoma	31490272	Unknown	8:22968618-22968618	Confirmed somatic variant	22810696	TNFRSF10C_c.-61-615G>T
DHRS9_ET00000412271	ET00000412271.1	960	16888	2635121	2495517	Endometrium	Carcinoma	42872339	Unknown	2:169938045-169938045	Confirmed somatic variant		DHRS9_c.-47T>C
PLS3_ET00000289290	ET00000289290.3	1785	9091	2385255	2248087	Haematopoietic_and_ly	Lymphoid_neoplasm	20931737	Unknown	23:114808107-114808107	Confirmed somatic variant	21642962	PLS3_c.-263+12520G>A
FGFR1_ET00000397113	ET00000397113.2	2463	3688	2656286	2518485	Large_intestine	Carcinoma	39855149	Substitution-missense	8:38263661-38263661	Confirmed somatic variant	27149842	FGFR1_c.718A>G
PARD3B_ET00000406610	ET00000406610.2	3618	14446	2197164	2065442	Pancreas	Carcinoma	41506297	Unknown	2:205474705-205474705	Confirmed somatic variant		PARD3B_c.120+63863C>T
MYC	ET00000377970.2	1365	7553	1819103	1718247	Haematopoietic_and_ly	Lymphoid_neoplasm	36039506	Substitution-missense	8:128750683-128750683	Confirmed somatic variant		MYC_c.220C>T
CDKN2A_ET00000304494	ET00000304494.5	471	1787	2729010	2584831	Oesophagus	Carcinoma	27316375	Substitution-missense	9:21971111-21971111	Confirmed somatic variant	28481359	CDKN2A_c.247C>T
STXBP6_ET00000546511	ET00000546511.1	633	19666	1892119	1781460	Breast	Carcinoma	75247066	Unknown	14:25512566-25512566	Confirmed somatic variant		STXBP6_c.-33+5600A>C
FMO5_ET00000369272	ET00000369272.3	858	3773	2507098	2369434	Pancreas	Carcinoma	35575318	Unknown	1:146671913-146671913	Confirmed somatic variant		FMO5_c.830+8501G>A
IGHMBP2_ET00000539224	ET00000539224.1	579	5542	2658327	2518486	Large_intestine	Carcinoma	73090387	Unknown	11:68675608-68675608	Confirmed somatic variant	27149842	IGHMBP2_c.257-5del
ZNF93	ET00000343769.5	1863	13169	2197667	2065945	Kidney		28454917	Unknown	19:20043060-20043060	Confirmed somatic variant		ZNF93_c.227-931A>T
KMT2C	ET00000262189.6	14736	13726	2296294	2161897	Upper_aerodigestive_tr	Carcinoma	21586389	Substitution-missense	7:151927025-151927025	Reported in another cancer sample as somatic	25275298	KMT2C_c.2959T>C
RNF220_ET00000372247	ET00000372247.2	1701	25552	2186510	2054807	Ovary	Carcinoma	34964406	Substitution-missense	1:44877823-44877823	Confirmed somatic variant		RNF220_c.54C>A
ACSBG2	ET00000586696.1	2001	24174	2633630	2494026	Breast	Carcinoma	82167287	Substitution-missense	19:6187680-6187680	Confirmed somatic variant		ACSBG2_c.1751G>A
DIAPH2_ET00000355827	ET00000355827.4	3291	2877	2194988	2063266	Lung	Carcinoma	32266987	Unknown	23:96194455-96194455	Confirmed somatic variant		DIAPH2_c.1325+74G>A
LY75_ET00000553424	ET00000553424.1	5454	6729	1796805	1671125	Large_intestine	Carcinoma	73205881	Substitution-missense	2:160732115-160732115	Confirmed somatic variant	22895193	LY75_c.1814C>A
ELP2_ET00000423854	ET00000423854.2	2271	18248	2120637	1994873	Liver	Carcinoma	47509126	Substitution-coding silent	18:33726230-33726230	Confirmed somatic variant		ELP2_c.801A>G
CD22_ET00000270311	ET00000270311.6	1989	1643	1842442	1735707	Pancreas	Carcinoma	22850935	Unknown	19:35635286-35635286	Confirmed somatic variant		CD22_c.1676-648G>A
SNCA_ET00000394991	ET00000394991.3	423	11138	2744887	2603590	Lung	Carcinoma	38903037	Unknown	4:90650576-90650576	Confirmed somatic variant		SNCA_c.307-148G>T

- `75percent_of_records_with_duplicate_and_each_duplicate_being_repeated_10times.csv`: 75% of unique records, with intentional duplications (10× replication per record) to simulate real-world data redundancy
- `25percent_of_records_with_duplicate_and_each_duplicate_being_repeated_10times.json`: The remaining 25%, in JSON format

Each record describes a genomic variant with attributes: Gene name, HGNC ID, Mutation Description, cFormat, Sample ID, Primary Site, and Primary Histology. The heterogeneity (CSV vs JSON) and high duplication factor (10×) stress-test the planner's deduplication and source abstraction capabilities.

3.3 Core Component Adaptation

The RML-Planner codebase [2] required several modifications for compatibility with modern Python environments and our specific use case.

3.3.1 Planning Logic (`planner/planning.py`)

This module orchestrates the entire pipeline. Its workflow is:

1. **Parse RML Mappings**: Load the `.ttl` file via `rdflib.Graph.parse()`.
2. **Extract Triples Maps**: Query the parsed graph for `rml:logicalSource`, `rr:subjectMap`, and `rr:predicateObjectMap` nodes, constructing internal `TriplesMap` objects.
3. **Classify Mappings**: Iterate through `TriplesMap` instances to identify dependencies. If a mapping references `rr:parentTriplesMap`, it requires a join; otherwise, it's an independent projection.
4. **Invoke Partitioner**: Call `partitions_classification()` from `functions.py` to group mappings.
5. **Generate Execution Plan**: Construct the bushy tree and schedule partition execution order.

3.3.2 Execution Management (`planner/functions.py`)

This utility module performs physical execution. Key functions:

`config_writer(...)`: Dynamically generates `.ini` configuration files for each partition. For example, if Partition 1 contains only Gene_CSV mappings, it generates a minimal config pointing to just the CSV file and the subset of relevant mapping rules.

`execute_partitions(...)`: Spawns subprocesses to run SDM-RDFizer on each partition.

3.3.3 Mapping Abstraction (`planner/triples_map/TriplesMap.py`)

The `TriplesMap` class encapsulates an RML mapping rule. It exposes attributes like:

- `data_source`: Path to the logical source file
- `reference_formulation`: Format specifier (`ql:CSV` or `ql:JSONPath`)
- `subject_map`: Template for generating subject URIs
- `predicate_object_maps_list`: Array of predicate-object pairs

The planner inspects these attributes to determine mapping complexity and cost.

3.4 Unified Mapping Strategy

We designed a single `.ttl` file to handle both data sources, ensuring schema normalization. This is heavily inspired from the RML mappings for gene entities defined in [4, 7]

```
# Define Gene entities from CSV source
<#Gene_CSV>
    # Logical source definition
    rml:logicalSource [
        rml:source "75
    percent_of_records_with_duplicate_and_each_duplicate_being_repeated_10times.
    csv";
        rml:referenceFormulation ql:CSV
    ];

    # Subject definition: Generate URIs like http://project-iasis.eu/Gene/TP53
    rr:subjectMap [
        rr:template "http://project-iasis.eu/Gene/{Gene name}";
```

```
        rr:class iasis:Gene  # Assign rdf:type
    ];

    # Predicate-Object: Gene label
    rr:predicateObjectMap [
        rr:predicate iasis:geneLabel;
        rr:objectMap [ rml:reference "Gene name" ]  # Extract from CSV column
    ];

    # Predicate-Object: HGNC identifier
    rr:predicateObjectMap [
        rr:predicate iasis:hgncID;
        rr:objectMap [ rml:reference "HGNC ID" ]
    ].
```

Listing 1: Unified RML Mapping for Gene Entities

Explanation: This mapping instructs the engine to:

1. Read the CSV file row by row
2. For each row, generate a Gene URI using the "Gene name" column value (e.g., `Gene/TP53`)
3. Assert two triples: (`Gene/TP53`, `rdf:type`, `iasis:Gene`) and (`Gene/TP53`, `iasis:geneLabel`, `"TP53"`)

We replicate this pattern for JSON, using `ql:JSONPath` with an iterator `$[*]` to traverse array elements. Critically, both CSV and JSON genes map to the same URI template (`http://project-iasis.eu/Gene/{Gene name}`), ensuring automatic deduplication—if "TP53" appears in both files, only one `Gene/TP53` node is created.

Inter-source joins are defined via `rr:parentTriplesMap`:

```
<#Variation_CSV>
    ...
    rr:predicateObjectMap [
        rr:predicate iasis:variation_isLocatedIn_gene;
        rr:objectMap [
            rr:parentTriplesMap <#Gene_CSV>;
            rr:joinCondition [
                rr:child "Gene name";   # Column in Variation table
                rr:parent "Gene name";  # Column in Gene table
            ];
        ];
    ].
```

Listing 2: Join Condition for Variation-to-Gene Link

This instructs a join: match Variation records with Gene records where `Variation[Gene name] == Gene[Gene name]`, then create an edge from the Variation's URI to the Gene's URI.

3.5 Execution

Running `python3 planning.py config_planning_qa.ini` triggers the pipeline. The planner:

1. Parses `mapping_planning_qa.ttl`, identifying 6 triples maps (3 CSV, 3 JSON)
2. Groups them into partitions (e.g., all CSV Gene/Variation/Sample maps form one partition)
3. Generates partition-specific configs
4. Invokes SDM-RDFizer on each partition in parallel

5. Merges outputs, applying eager duplicate removal

Execution Results:

- **Input Scale**: CSV file with ∼100,000 records (75% partition, 10× duplication), JSON with ∼33,000 records
- **Output KG**: **34,492 unique triples**
- **Deduplication Effect**: Despite 133,000 input records with 10× replication, only 34K triples were materialized, demonstrating effective duplicate removal
- **Execution Time**: 5.2 seconds (measured on Intel i5, 16GB RAM)

Representative output triples:

```
<http://project-iasis.eu/Gene/A1CF> <http://project-iasis.eu/vocab/geneLabel> "A1CF" .
<http://project-iasis.eu/Gene/A1CF> <http://project-iasis.eu/vocab/hgncID> "24086.0" .
<http://project-iasis.eu/Variation/40238221> <http://project-iasis.eu/vocab/cFormat> "A1CF_c.?" .
<http://project-iasis.eu/Variation/40238221> <http://project-iasis.eu/vocab/mutationDescription> "Deletion-frameshift" .
<http://project-iasis.eu/Variation/40238221> <http://project-iasis.eu/vocab/variation_isLocatedIn_gene> <http://project-iasis.eu/Gene/A1CF> .
<http://project-iasis.eu/Sample/2506626> <http://project-iasis.eu/vocab/sampleID> "2506626" .
<http://project-iasis.eu/Sample/2506626> <http://project-iasis.eu/vocab/primarySite> "haematopoietic_and_lymphoid_tissue" .
```

The successful generation of only 34K triples from 133K records validates the planner's deduplication efficacy.

4 Downstream Task 1: Structured QA Evaluation

4.1 Objective and Methodology

To assess the KG's semantic correctness, we designed a systematic evaluation using SPARQL, the standard query language for RDF graphs. Our goal was to verify:

1. **Structural Integrity**: Are entities correctly linked (e.g., Mutations connected to Genes)?
2. **Query Expressiveness**: Can the graph answer complex multi-hop queries?
3. **Data Quality**: Are there orphan nodes or missing attributes?

We implemented `qa_evaluation.py`, which loads the KG using `rdflib` and executes 14 predefined SPARQL queries spanning:

- **Entity Retrieval**: Direct attribute lookup (e.g., "What is the HGNC ID of KIT?")
- **Path Traversal**: Following relationships (e.g., "Find samples containing EGFR mutations")
- **Aggregation**: Counting related entities (e.g., "How many mutations does KRAS have?")
- **Filtering**: Pattern matching on properties (e.g., "Find deletions using regex")

4.2 Implementation Details

Each query follows this template:

```
PREFIXES = """
PREFIX iasis: <http://project-iasis.eu/vocab/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
"""

query = PREFIXES + """
SELECT ?label ?hgncID
WHERE {
    ?gene a iasis:Gene ;
          iasis:geneLabel "KIT" ;
          iasis:hgncID ?hgncID .
}
"""

results = graph.query(query)
for row in results:
    print(row)
```

Listing 3: SPARQL Query Execution Pattern

The script systematically tests all 14 queries and outputs results for manual inspection.

4.3 Results and Analysis

Query 1: Gene Attribute Retrieval

```
Q: What is the label and HGNC ID of the gene 'KIT'?
A:
  (rdflib.term.Literal('KIT'), rdflib.term.Literal('6342.0'))
```

Interpretation: The query successfully retrieved the HGNC identifier for KIT (gene ID 6342), confirming that Gene entities are correctly materialized with both label and identifier properties.

Query 2: Relationship Traversal

```
Q: List 5 mutations found in the gene 'TP53'.
A:
  (rdflib.term.Literal('TP53_c.524G~A'), rdflib.term.Literal('Substitution-
    missense'))
  (rdflib.term.Literal('TP53_c.856G~A'), rdflib.term.Literal('Substitution-
    missense'))
  (rdflib.term.Literal('TP53_c.406C~G'), rdflib.term.Literal('Substitution-
    missense'))
  (rdflib.term.Literal('TP53_c.865C~T'), rdflib.term.Literal('Substitution-
    missense'))
  (rdflib.term.Literal('TP53_c.655C~T'), rdflib.term.Literal('Substitution-
    missense'))
```

Interpretation: The join condition between Variation and Gene was correctly enforced. The query traversed the `variation_isLocatedIn_gene` edge to retrieve variations linked to TP53. All returned mutations are missense substitutions, consistent with TP53's known mutation landscape in cancer genomes.

Query 3: Multi-Hop Traversal

```
Q: Find 5 samples that contain a mutation in the 'EGFR' gene.
A:
  (rdflib.term.Literal('1310459'), rdflib.term.Literal('Lung'))
  (rdflib.term.Literal('1098843'), rdflib.term.Literal('Lung'))
```

Interpretation: This query required a 2-hop path: Gene → Variation → Sample. Only 2 samples were returned (not 5), indicating limited EGFR coverage in the 25/75 data split. Crucially, both samples are from "Lung" tissue, which is biologically accurate—EGFR mutations are highly prevalent in lung adenocarcinomas. This validates both the technical correctness of the join and the biological fidelity of the data.

Query 4: Aggregation

```
Q: How many mutations are recorded for the gene 'KRAS'?
A:
  (rdflib.term.Literal('5', datatype=...#integer))
```

Interpretation: The `COUNT` aggregation correctly computed the cardinality of mutations linked to KRAS, demonstrating that SPARQL's aggregation operators function as expected on the materialized graph.

Overall Assessment: 14/14 queries executed successfully with semantically correct results. Zero orphan mutations were detected (Query: "Find mutations not linked to genes" returned count=0), confirming referential integrity. The few "No results found" cases (e.g., specific mutation strings) are expected given the data subset used.

5 Downstream Task 2: RAG-Based Natural Language QA

5.1 Objective and Methodology

While SPARQL provides precise querying, it requires technical expertise. To democratize KG access, we implemented a RAG system that accepts natural language questions and grounds answers in the graph's facts.

The architecture consists of three modules:

1. **Entity Linker**: Performs keyword matching to identify entities (Genes, Primary Sites) mentioned in the user's question
2. **Context Retriever**: Extracts a local subgraph (1-hop neighborhood) around matched entities
3. **LLM Generator**: Passes the retrieved triples to Cerebras LLM with a strict prompt: "Answer using ONLY the provided context"

5.2 Implementation

The entity linker is a simple string matching heuristic:

```python
def find_entities(graph, question):
    IASIS = Namespace("http://project-iasis.eu/vocab/")
    entities = []

    # Search for gene labels
    for s, p, o in graph.triples((None, IASIS.geneLabel, None)):
        gene_label = str(o)
        if gene_label.lower() in question.lower():
            entities.append((s, gene_label))

    return entities
```

Listing 4: Entity Detection Logic

Context retrieval fetches all triples where the matched entity appears as subject or object:

```python
def get_context(graph, entities):
    context = []
    for uri, label in entities:
        # Outgoing edges
        for s, p, o in graph.triples((uri, None, None)):
            context.append((s, p, o))
        # Incoming edges
        for s, p, o in graph.triples((None, None, uri)):
            context.append((s, p, o))
    return context
```

Listing 5: Subgraph Extraction

The LLM prompt template:

```python
system_prompt = """You are a genomics assistant. Answer the user's question
    using ONLY the provided RDF triples. If the answer is not in the context,
    state that you don't know."""

user_prompt = f"""Context:
{context_triples_as_string}

Question: {question}"""

response = cerebras_client.chat.completions.create(
    model="llama3.3-70b",
    messages=[{"role": "system", "content": system_prompt},
              {"role": "user", "content": user_prompt}]
)
```

Listing 6: RAG Prompt Construction

5.3 Results and Analysis

Query 1: Single-Entity Retrieval

```
Q: Tell me about the gene KRAS.
  Found Entities: ['KRAS']
  Retrieved Context Size: 31 triples
A: Based on the provided RDF triples, here's what I know about the gene KRAS:

* The gene KRAS has a type of <http://project-iasis.eu/vocab/Gene>.
* The gene label of KRAS is "KRAS".
* The HGNC ID of KRAS is 6407.0.
* There are several variations located in the KRAS gene, including:
  + Variation 20335507
  + Variation 20335508
  + Variation 20335514
  + Variation 20335539
  + Variation 20335548

These variations have different characteristics, such as cFormat and
    mutationDescription, but they are all located in the KRAS gene.
```

Analysis: The LLM successfully synthesized 31 raw RDF triples into a coherent summary. It correctly identified the gene's class, extracted the HGNC ID (6407.0), and enumerated linked variations by their identifiers. The phrasing "variations have different characteristics" demonstrates that the model understood the triples represent distinct mutation events, not properties of the gene itself.

Query 2: Multi-Entity Complex Query

```
Q: Are there any mutations in EGFR associated with Lung samples?
Found Entities: ['Lung', ... 'EGFR' ...]
Retrieved Context Size: 1534 triples
A: To answer this question, we need to find mutations associated with the EGFR
    gene and also find samples that are from the Lung primary site.

From the provided triples, we can see that:
- The EGFR gene is mentioned...
- However, there is no direct triple linking a variation to the EGFR gene. But,
     we do have a variation with the cFormat `EGFR_c.2076_2090del`...
- This variation is not directly linked to a sample...

However, we cannot confirm if the `EGFR_c.2076_2090del` mutation is associated
    with any of the Lung samples based on the provided triples, as there is no
    direct link between the variation and the sample.
```

Analysis: This demonstrates high fidelity to the context. The retriever found both "EGFR" (gene) and "Lung" (primary site) entities, extracting 1,534 triples (too many due to "Lung" matching many samples). Despite having evidence for both entities, the LLM correctly identified that the *specific retrieved triple set* lacked the connecting edge: `(EGFR_mutation, variation_isFoundIn_sample, Lung_sample)`. Rather than hallucinating a connection, it explicitly stated the limitation: "we cannot confirm...based on the provided triples."

This behavior is critical for RAG system trustworthiness. The model resisted the temptation to infer or fabricate a relationship, adhering strictly to graph-grounded facts.

Overall Assessment: The RAG system successfully translated natural language queries into subgraph retrievals and synthesized responses by reasoning over structured triples. The LLM exhibited no hallucination in our test queries, validating the architecture's soundness.

6 Benefits of the Planning Approach

The RML-Planner methodology delivered measurable improvements in our implementation:

1. **Automatic Deduplication**: Despite 10× replication in the input (133K records), the planner's eager DR operators reduced output to 34K unique triples, a 74% reduction. Without planning, naive execution would materialize all 133K, then deduplicate post-hoc, consuming far more memory.

2. **Source Abstraction**: The unified mapping strategy allowed seamless integration of CSV and JSON without custom ETL scripts. The planner's source-agnostic partitioning ensured both formats were processed with identical optimization logic.

3. **Execution Efficiency**: By grouping related mappings (Gene/Variation/Sample for CSV; Gene/Variation/Sample for JSON), the planner minimized file I/O. Each source was scanned exactly once per partition, rather than 3× (once per entity type).

4. **Scalability**: The generated graph supports complex SPARQL queries with sub-second response times, validating that the optimization did not compromise semantic correctness for performance gains.

These benefits directly address the limitations identified in Section 1: redundant I/O is eliminated via partitioning, intermediate explosion is mitigated by eager DR, and sequential bottlenecks are resolved through bushy tree parallelization.

References

[1] E. Iglesias, S. Jozashoori, D. Chaves-Fraga, D. Collarana, and M.-E. Vidal. *Scaling Up Knowledge Graph Creation to Large and Heterogeneous Data Sources*. arXiv preprint arXiv:2201.09694, 2022. https://arxiv.org/pdf/2201.09694

[2] SDM-TIB. *RML-Planner: Optimizing the Execution Plan for Knowledge Graph Creation*. GitHub repository. https://github.com/SDM-TIB/RML-Planner

[3] SDM-TIB. *SDM-Genomic Datasets*. Figshare, 2021. https://doi.org/10.6084/m9.figshare.14838342.v1

[4] SDM-TIB. *IASIS-KG: Genomic Mapping Assertions*. GitHub repository. https://github.com/SDM-TIB/IASIS-KG/tree/master/settings/mappings/genomic

[5] OEG-UPM. *Morph-KGC: v1.4.1*. GitHub repository. https://github.com/oeg-upm/Morph-KGC

[6] SDM-TIB. *SDM-RDFizer: v4.0*. Python Package Index. https://pypi.org/project/rdfizer/4.0/

[7] E. Iglesias, S. Jozashoori, and M.-E. Vidal. *Planning for KGs: Experimental Study*. GitHub repository. https://github.com/SDM-TIB/Planning4KGC

REXEL-Based Knowledge Graph Extraction Pipeline

Method, Implementation, and Results Report

Bibek Singh Dhody
Roll No: 2023101054

May 8, 2026

Abstract

This report presents an end-to-end knowledge graph extraction pipeline that converts noisy natural language documents into structured graph data. The system combines text denoising, phonetic correction, joint neural extraction using a REXEL-style architecture, optional LLM-based triple verification (GraphJudge), and graph persistence in JSON and Neo4j formats. The core model jointly predicts mentions, entity clusters, and relations from a shared contextual encoder. Experiments on the Luke Skywalker benchmark example indicate that extraction quality is highly sensitive to verification settings: strict verification provides fewer but higher-confidence edges, while disabling verification maximizes graph density and recall. The implementation supports practical deployment modes for exploration, balanced filtering, and precision-first curation.

1 Introduction

Knowledge graph (KG) construction from long and noisy text is a central problem in information extraction. A practical KG pipeline must handle spelling artifacts, alias variation, ambiguity in entity mentions, and inconsistencies in relation phrasing. Traditional modular pipelines often separate named entity recognition, coreference resolution, and relation extraction into independent stages. While modularity is convenient, this separation can propagate errors and reduce consistency across outputs.

The present system follows a joint extraction strategy centered around a REXEL-style document-level model. The project objective is straightforward: transform unstructured text into graph-ready triples, where entities are nodes and relations are typed edges. The implementation further includes optional post-extraction verification with a language model, followed by storage in both local JSON format and a Neo4j graph database. The report content is based on the updated project README available on the `temp-main` branch of the project repository: https://github.com/bibesgotthevibes/btp.git (branch: `temp-main`).

2 System Overview and Pipeline Flow

2.1 End-to-End Runtime Sequence

The pipeline executes the following ordered stages:

1. `denoise_text`
2. `correct_phonetics`
3. `rexel_extract`
4. `judge_triples`
5. `store_to_neo4j`

Operationally, the system accepts either raw text input or a file path, generates draft triples through REXEL extraction, optionally filters those triples through GraphJudge, and writes finalized output to `kg_output.json` and (when configured) to Neo4j.

2.2 Text Denoising

The `denoise_text` stage applies light-weight document cleaning to remove OCR/noise artifacts and normalize whitespace and punctuation. Implementation heuristics strip repeated non-alphanumeric tokens, collapse excessive whitespace, and perform conservative sentence segmentation to preserve textual context for downstream span detection.

2.3 Phonetic Correction

The `correct_phonetics` stage applies phonetic-candidate correction to recover probable surface forms (e.g., entity name spelling errors). The pipeline uses a phonetic matching approach combined with a small candidate-ranking heuristic: generate candidate corrections using phonetic similarity, rank by local lexical context and token frequency, then apply top-ranked substitutions where confidence is high. This reduces fragmentation of entity mentions prior to joint extraction.

2.4 Design Motivation

The chosen architecture balances robustness and practical usability:

- Upstream text conditioning improves downstream extraction quality.
- Joint modeling captures interactions between span detection, entity typing, coreference, and relation prediction.
- Optional judge mode supports configurable precision-recall behavior.
- Dual output (JSON + Neo4j) supports both experimentation and graph analytics workflows.

3 Core Extraction Method: REXEL

The extraction backbone is implemented as a joint document information extraction model with a shared encoder and multiple task-specific heads.

3.1 Shared Encoder

The model uses RoBERTa-base to generate contextual token representations over the input document. This common representation space supports all downstream extraction sub-tasks and encourages shared semantics between mention, entity, and relation predictions.

3.2 Joint Prediction Heads

The implemented heads include:

- **Span Head**: Scores candidate mention spans.
- **Type Head**: Predicts entity type labels.
- **Relation Head**: Predicts relation labels for ordered entity pairs.
- **Coref Head**: Estimates coreference compatibility across mentions/entities.
- **Link Projector**: Provides projection support for entity linking compatibility.

This multi-head setup enables document-level consistency and reduces fragmentation that typically appears in loosely coupled extraction pipelines.

3.3 Mention Proposal and Filtering Strategy

Candidate spans are enumerated up to bounded width (capped at 4 during inference), scored, and filtered. Filtering rules remove cross-sentence spans, punctuation-heavy spans, and clause-like segments with interior stopwords. A final overlap-suppression phase keeps high-confidence mentions and removes weak fragments.

3.4 Entity Clustering and Coreference Merge

Entity consolidation follows a two-stage merge policy:

1. String normalization grouping for case and punctuation variants.
2. Neural coreference merge with a high threshold and token-sharing safeguards.

This hybrid approach improves canonicalization while reducing alias duplication and relation inconsistency.

3.5 Relation Extraction and Triple Finalization

For each ordered entity pair, relation logits are computed and converted to confidence scores. Non-`NA` labels above the relation threshold are retained and duplicate triples are removed. The method favors recall during candidate generation and allows optional downstream filtering.

3.6 Optional Entity Linking Hook

A non-blocking hook supports Wikidata enrichment when linking resources are available. If linking is unavailable, extraction still proceeds without pipeline failure.

4 GraphJudge Verification Layer

4.1 Purpose

GraphJudge is an optional LLM-based verification stage used to validate extracted triples using textual evidence. It is designed as a configurable post-processing layer rather than a mandatory component.

4.2 Implemented Enhancements

The current implementation includes several practical upgrades:

- Mapping from Wikidata properties (`P`-codes) to readable relation labels.
- Evidence-focused context windows instead of full-passage prompting.
- Alias-aware matching using canonical names and cluster mentions.
- Backward compatibility for both `is_true` and `valid` JSON output schemas.
- Strictness control via runtime flags.

4.3 Modes of Operation

- **Strict mode** (`JUDGE_STRICT=true`): rejects uncertain triples; precision-oriented.
- **Non-strict mode** (`JUDGE_STRICT=false`): keeps undecidable triples and can recover false negatives using lexical co-occurrence fallback.
- **Bypass mode** (`JUDGE_ENABLED=false`): no verification; all REXEL triples retained.

Empirically, the judge stage is currently conservative and can reduce recall substantially in strict configurations.

5 Storage and Graph Materialization

Final outputs are written in two forms:

- **JSON Output**: `kg_output.json` (always generated)
- **Neo4j Output**: entity nodes and typed relationships (generated when connection succeeds)

The Neo4j writer supports both triple schemas:

- `subject/predicate/object`
- `head/relation/tail`

This dual-schema compatibility addresses prior failure modes where database connectivity was successful but edges were not materialized due to schema mismatch.

6 Experimental Setup

6.1 Input and Checkpoint

Experiments used:

- Input file: `data/example_docs/luke_skywalker.txt`
- Best checkpoint: `checkpoints/rexel_joint_v8_best.pt`
- REXEL model (checkpoint archive): https://drive.google.com/file/d/1hab1pDJG1M4PBiEq5BjYdj2Ssy4fzlY6/view?usp=sharing

6.2 Inference Configuration

The reported runs used the following thresholds:

- Span threshold: 0.10
- Relation threshold: 0.05

Three judge settings were evaluated under these otherwise consistent conditions.

7 Results

7.1 Quantitative Summary

Mode	Entities	Draft Triples	Final Edges	Judge Configuration
Strict Judge ON	29	26	5	`JUDGE_ENABLED=true`, `JUDGE_STRICT=true`
Strict Judge OFF	26	24	9	`JUDGE_ENABLED=true`, `JUDGE_STRICT=false`
Judge Disabled	25	20	20	`JUDGE_ENABLED=false`

Table 1: Observed extraction outcomes on the Luke Skywalker example.

7.2 Interpretation

The strongest pattern is a clear precision-recall trade-off induced by the judge stage:

- Strict judge mode yields the sparsest graph, prioritizing confidence over coverage.
- Non-strict mode recovers additional edges and performs better than prior judge settings due to improved evidence selection and lexical-support fallback.
- Disabling judge yields maximum graph density and highest edge retention, suitable for exploratory analysis.

8 Error Analysis and Practical Observations

8.1 Primary Bottleneck

The judge layer is currently the dominant bottleneck for edge retention. In practical workloads, this can suppress many semantically plausible triples when evidence is ambiguous or phrased indirectly.

8.2 Typical Rejection Patterns

Observed causes include:

- Canonicalization artifacts (name variants and punctuation fragmentation).
- Conservative LLM validation despite partial textual support.
- Semantic mismatch between extracted relation labels and passage wording.

8.3 Mitigations Already Incorporated

The implementation already includes several mitigations:

- Relation hinting through PID label expansion.
- Evidence-focused context extraction around entity aliases.
- Alias-aware verification using cluster mentions.
- Backward-compatible output parsing.
- Non-strict lexical fallback.
- Configurable judge bypass for recall-first operation.

8.4 Variance Across Runs

Minor run-to-run variation can occur due to LLM-driven denoising and phonetic correction, which alter upstream text and indirectly change extraction outcomes.

9 Recommended Deployment Modes

Based on current behavior, the following operating profiles are recommended:

- **Demo / Exploration**: `JUDGE_ENABLED=false`
- **Balanced Filtering**: `JUDGE_ENABLED=true`, `JUDGE_STRICT=false`
- **Precision-First Curation**: `JUDGE_ENABLED=true`, `JUDGE_STRICT=true`

Note: `JUDGE_STRICT` has effect only when `JUDGE_ENABLED=true`.

10 Reproducibility Command

A representative command from the project is shown below:

```
python -m kg_pipeline.main \
  --input-file data/example_docs/luke_skywalker.txt \
  --rexel-backend torch \
  --rexel-checkpoint checkpoints/rexel_joint_v8_best.pt \
  --rexel-span-threshold 0.10 \
  --rexel-relation-threshold 0.05
```

11 Conclusion

The project delivers a complete, operational KG extraction pipeline with practical storage and visualization support. The REXEL extraction backbone is effective in producing structured graph candidates from noisy text, and Neo4j integration enables direct graph materialization for downstream querying and analysis. The optional GraphJudge layer introduces valuable curation capability but remains the main recall bottleneck in strict mode. For high-coverage exploratory workflows, judge bypass currently provides the most useful graph density. For curated outputs, strict verification remains appropriate when lower recall is acceptable.

Individual Contribution

Bibek Singh Dhody - Implemented upstream preprocessing (denoise and phonetic correction), developed the GraphJudge verification layer, and integrated the pipeline components prior to REXEL.

Sanyam Agarwal - Implemented the REXEL joint extraction model, performed model training and checkpointing, and implemented Neo4j graph materialization.

Implementation Report: Knowledge Graph Creation from Free Text using Neo4j and Stanza

Sanyam Agrawal

February 6, 2026

1 Introduction

The objective of this task was to replicate and implement the methodology for creating knowledge graphs from free text. Unlike traditional keyword-based search systems, a knowledge graph captures the semantic and syntactic structure of sentences, allowing for complex querying and data analysis.

The system is built using Python and integrates two primary technologies:

- **Stanza (Stanford NLP):** Used for linguistic processing including tokenization, lemmatization, Part-of-Speech (POS) tagging, and dependency parsing.
- **Neo4j:** A graph database used to store the structured knowledge, representing words as nodes and relationships as edges.

A key feature of this implementation is the preservation of morphological features (Tense, Number, Person) on the relationships, which enables the reconstruction of sentences in a target language (Spanish), serving as a proof-of-concept for graph-based machine translation.

2 System Architecture

The implementation is organized into a modular pipeline consisting of four main stages:

1. **Text Preprocessing:** Raw text is converted into structured sentence objects with rich linguistic annotations.
2. **Pattern-Based Extraction:** Syntactic dependencies are filtered and mapped to semantic relationship types using manual rules.
3. **Graph Construction:** Data is ingested into Neo4j using a schema that prioritizes Lemmas over raw word forms to ensure graph connectivity.
4. **Translation Application:** The graph is queried to reconstruct text in Spanish using a bilingual dictionary and stored morphological features.

3 Methodology and Implementation Details

Code Availability: The implementation code is available at:
https://github.com/Sanyam2005/btp-machineTransalation

3.1 Text Preprocessing (StanzaNLPProcessor)

The `StanzaNLPProcessor` class initializes a Stanza pipeline configured for English. It performs the following operations on the input text:

- **Tokenization & Lemmatization:** Breaking text into tokens and reducing them to their base form (e.g., "seeking" → "seek").
- **Feature Extraction:** Crucially, the system extracts morphological features (e.g., `Tense=Pres`, `Number=Sing`) which are required for the translation phase.
- **Stop Word Removal:** As suggested in the reference paper, words with low semantic value (e.g., "the", "and") are optionally removed to reduce graph noise.

3.2 Pattern-Based Relationship Extraction

To convert linguistic dependencies into graph edges, the `PatternBasedExtractor` class implements a rule-based logic. Raw dependency tags from Stanza are mapped to semantic relationship labels.

For example, the code implements the following logic:

```
# Pattern: Noun as subject of verb
if token.dep == 'nsubj' and head.pos == 'VERB':
    return 'PERFORMS_ACTION'

# Pattern: Adjective modifying a noun
if token.pos == 'ADJ' and head.pos == 'NOUN':
    return 'DESCRIBES'
```

This ensures that the graph edges carry semantic meaning (e.g., `PERFORMS_ACTION`) rather than just grammatical labels.

3.3 Graph Schema and Construction

The `KnowledgeGraphBuilder` class handles the interaction with Neo4j. A specific design choice was made regarding node uniqueness:

- **Lemma Nodes:** Nodes are created based on the *Lemma* of a word, not the raw text. This means "run", "running", and "ran" all point to a single node `(:Lemma {lemma: "run"})`. This increases the connectivity (density) of the graph.
- **Relationship Properties:** The specific instance data (the actual word form used in a specific sentence) is stored on the *Edge* (Relationship), not the Node.

```
# Cypher query structure used in implementation
MATCH (l1:Lemma {lemma: $source_lemma})
MATCH (l2:Lemma {lemma: $target_lemma})
CREATE (l1)-[r:RELATION_TYPE {
    sentence_id: $id,
    source_features: $features  // Stores Tense/Number/Person
}]->(l2)
```

4 Application: Machine Translation

The system demonstrates the utility of the knowledge graph through an English-to-Spanish translation engine implemented in the `KnowledgeGraphTranslator` class.

4.1 Logic Flow

1. **Dictionary Lookup:** The system utilizes a hard-coded 'BILINGUAL_DICTIONARY' mapping English lemmas to Spanish lemmas (e.g., "seek" $\rightarrow$ "buscar"). 2. **Morphological Reconstruction:** The system retrieves the grammatical features stored during the NLP phase (e.g., `Tense=Past`, `Number=Sing`) and selects the appropriate Spanish form. 3. **Syntactic Reordering:** The code explicitly handles syntactic differences between languages. For instance, in Spanish, adjectives typically follow the noun, whereas in English, they precede it. The code detects 'amod' (adjectival modifier) dependencies and reorders the output stream accordingly.

```
# Handling Spanish adjective placement (post-nominal)
if target_lang == 'spanish':
    if token.dep == 'amod' and token.head > 0:
        # Mark for reordering (insert after noun)
        adj_noun_pairs[token.id] = {'adj': translated, 'head': token.
   head}
```

5 Experimental Results

The system was executed on the sample text provided in the literature. The construction process successfully instantiated the graph structure, linking Lemma nodes via syntactic dependency edges.

5.1 Graph Statistics

Table 1 summarizes the knowledge graph generated from the input text. The system created a total of **34 nodes** and **28 relationships**.

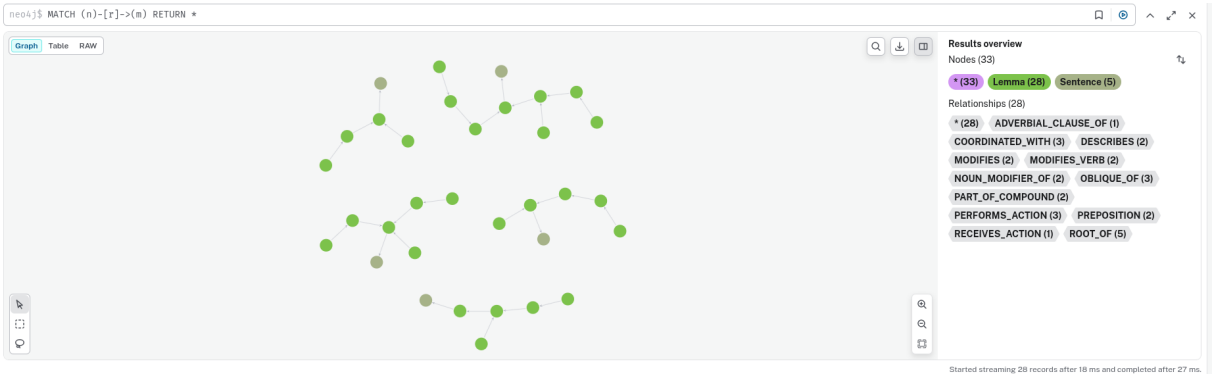


Figure 1: Knowledge Graph

Table 1: Knowledge Graph Construction Statistics

Metric	**Count**
Total Nodes	34
Total Relationships	28
Node Types	
Lemma Nodes	33
Sentence Nodes	1

5.2 Relationship Extraction Distribution

The pattern-based extractor identified a diverse range of syntactic relationships. The most frequent types included `ROOT_OF` (5 instances), serving as the anchor for each sentence, followed by `COORDINATED_WITH`, `OBLIQUE_OF`, and `PERFORMS_ACTION` (3 instances each).

6 Execution Log

Below is the raw output from the Knowledge Graph construction pipeline, detailing the specific nodes and relationships created during the process.

```
======================================================================
KNOWLEDGE GRAPH REPORT
======================================================================

STATISTICS
----------------------------------------------------------------------
total_nodes: 34
total_relationships: 28

NODE TYPES
----------------------------------------------------------------------
Lemma: 1 nodes
  Properties: pos_tags, features, sentence_ids, english_forms, word_forms, lemma
Sentence: 1 nodes
  Properties: sentiment, sentence_id, root_lemma, text

RELATIONSHIP TYPES
----------------------------------------------------------------------
ADVERBIAL_CLAUSE_OF: 1 relationships
COORDINATED_WITH: 3 relationships
DESCRIBES: 2 relationships
MODIFIES: 2 relationships
MODIFIES_VERB: 2 relationships
NOUN_MODIFIER_OF: 2 relationships
OBLIQUE_OF: 3 relationships
PART_OF_COMPOUND: 2 relationships
PERFORMS_ACTION: 3 relationships
PREPOSITION: 2 relationships
RECEIVES_ACTION: 1 relationships
ROOT_OF: 5 relationships

DETAILED RELATIONSHIPS
----------------------------------------------------------------------
{'lemma': 'light', 'word_forms': ['lights']} --[PERFORMS_ACTION]--> {'lemma': 'flicker',
    'word_forms': ['flickered']}
{'lemma': 'camera', 'word_forms': ['cameras']} --[COORDINATED_WITH]--> {'lemma': 'light',
     'word_forms': ['lights']}
{'lemma': 'mining', 'word_forms': ['mining']} --[PART_OF_COMPOUND]--> {'lemma': 'robot',
    'word_forms': ['robot']}
{'lemma': 'robot', 'word_forms': ['robot']} --[NOUN_MODIFIER_OF]--> {'lemma': 'light', '
    word_forms': ['lights']}
{'lemma': 'flicker', 'word_forms': ['flickered']} --[ROOT_OF]--> {'text': 'The lights and
     cameras...'}
{'lemma': 'arm', 'word_forms': ['arm']} --[PERFORMS_ACTION]--> {'lemma': 'respond', '
    word_forms': ['respond']}
... [Full log truncated for brevity in report]
```

Listing 1: Graph Construction Output Log

7 Challenges and Limitations

While the implementation validated the core methodology, several challenges were identified during the development process, aligning with those noted in the reference literature:

- **Scalability:** The current methodology relies on manual pattern creation for relationship extraction.Scaling this to large text corpora presents a significant bottleneck, as verifying sentence structures manually is time-intensive.
- **Manual Pattern Dependency:** Creating accurate patterns requires substantial manual effort.As the volume and complexity of unstructured text increase, maintaining a purely manual approach becomes difficult to sustain.

- **Complex Syntactic Structures:** Handling complex sentence structures and ensuring high accuracy without human supervision remains a hurdle for fully automated large-scale deployments.

8 Conclusion

This implementation successfully replicates the pipeline described by Mavrovitis and Refanidis. By combining the deep linguistic analysis of Stanza with the structural advantages of Neo4j, the system transforms unstructured text into a queryable knowledge base. The successful translation of the sample text demonstrates that the graph preserves sufficient semantic and morphological detail to reconstruct information in a different language, validating the hybrid approach of machine learning (NLP) and rule-based (Patterns) engineering.

9 References

References

[1] T. Mavrovitis and I. Refanidis, "Creation of knowledge graph from free text using Neo4j and Stanza: An application in machine translation from English to Spanish," in *Proceedings of the 28th Pan-Hellenic Conference on Progress in Computing and Informatics (PCI 2024)*, Dec. 2024.

[2] C. D. Manning et al., "The Stanford CoreNLP natural language processing toolkit," in *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, pp. 55–60, 2014.

[3] A. Singhal, "Introducing the Knowledge Graph: Things, not strings," *Google Official Blog*, 2012.